



Compiler-Induced Feature Drift in Static Malware Analysis

Christoffer Larsson

This thesis is submitted to the Faculty of Faculty at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Master of Science in Computer Science. The thesis is equivalent to Weeks weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:

Author(s):

Christoffer Larsson

E-mail: chla23@student.bth.se

University advisor:

Title Firstname Lastname

Department of Department

Faculty of Faculty
Blekinge Institute of Technology
SE-371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Most readers will turn first to the abstract of your thesis. Use it as an opportunity to spur the reader's interest. The abstract should highlight the main points of your work, especially the thesis' problem statement, methods, findings, and conclusions. However, the abstract does not need to cover every aspect of your work. The main objective is to give the reader a good idea of what the thesis is about.

The abstract should be completed towards the end when you are able to overview your project as a whole. It is nevertheless a good idea to work on a draft continuously. Writing a good abstract can be difficult since it should only include the most important points of your work. But this is also why working on your abstract can be so useful – it forces you to identify the key elements of your degree project.

Structured abstracts have several advantages for authors and readers. They help readers to quickly find information in an abstract and also guide authors in summarizing the content of their manuscripts precisely. Below you find the main components of a structured abstract.

Background. ...

Objectives. ...

Methods. ...

Results. ...

Conclusions. ...

Keywords: Up to 5 keywords, at most 2 of these should appear in the title. Starts 1 line below the abstract.

DRAFT

Acknowledgments

Here you can add your acknowledgements.

DRAFT

Contents

Abstract	i
Acknowledgments	iii
1 Preface	1
2 Introduction	3
2.1 Malware Analysis	3
2.1.1 Machine Learning	4
2.1.2 Evasive Malware	6
2.2 Problem Statement	6
2.2.1 Aim	7
2.3 Thesis Structure	8
3 Related Work	9
3.1 Feature Drift & Malware Evasion	9
3.1.1 Evasive Malware	9
3.1.2 Code Transformation	9
3.1.3 Source Code Obfuscation	11
3.2 Malware Analysis Benchmarking Issues	11
3.2.1 Implications	12
3.3 Research Gap	13
4 Method	15
4.1 Research Questions	15
4.2 Experiment Design	15
4.2.1 Design Overview	16
4.3 C Modules	17
4.3.1 Compiled Programs	17
4.4 Compiler Flags	19
4.4.1 Excluded Flags	20
4.4.2 Included Flags	21
4.4.3 Flag Configurations	21
4.5 Tigress Transformations	22
4.6 Compilation Pipeline	23
4.6.1 Tigress	23
4.7 Data Collection Pipeline	24
4.8 Metric Usage	24

4.8.1	RQ1	25
4.8.2	RQ2	25
4.8.3	RQ3	26
4.8.4	Relation to RQ4	26
4.9	Validity, Reliability, and Limitations	27
4.9.1	Limitations	27
4.9.2	Internal Validity	28
4.9.3	External Validity	29
5	Results and Analysis	31
5.1	RQ1	32
5.1.1	Analysis	32
5.2	RQ2	34
5.2.1	Analysis	34
5.3	RQ3	36
5.3.1	Analysis	36
5.4	RQ4	39
5.4.1	Model Convergence	39
5.4.2	Model Prediction Estimates	39
5.4.3	Analysis	40
5.5	Summary	41
6	Discussion	43
6.1	Answers to Research Questions	43
6.1.1	RQ1	43
6.1.2	RQ2	44
6.1.3	RQ3	44
6.1.4	RQ4	45
6.2	Opcode 3-gram Stability	47
6.3	Relation to Related Work	47
6.3.1	Gibert et al	47
6.3.2	Ren et al	48
6.3.3	Research Gap	48
6.4	Implications	50
6.4.1	Robustness Evaluation	51
6.4.2	Dataset Construction	51
6.5	Trustworthiness	52
6.5.1	Limitations	52
7	Conclusions and Future Work	55
7.1	Future Work	55
	References	57
A	T1005: File Extensions	63
A	Fractional Factorial Design Table	65

B	Code Examples	67
B.1	C Code	67
B.2	Qspectre-load: C code shown in Appendix B.1	67
B.3	/homeparams: C code shown in Appendix B.1	67
B.4	/GH & /Gh: C code shown in B.1	68
C	Blacklisted Functions	69
D	Tooling and Software Versions	73
D.1	Software	73
D.2	Python Libraries	73

DRAFT

Chapter 1

Preface

- Results: objective results, i.e., the data you collected without analysis and interpretation
- Analysis: an objective analysis of the results that is based solely on the collected data
- Discussion: interpretation of and reasoning about the results and analysis within the context of the body of knowledge (external to your thesis)

Table 1.1 provides suggestions for a range of page lengths for each thesis chapter. Please note that these are rough estimates for your orientation. For a thesis at master level, the chapter lengths can be expected in the higher ranges, and for a thesis at bachelor level in the lower ranges. For a thesis at master level, the chapter lengths should normally not fall below the minimum length estimates, though.

The complete thesis text, excluding preliminaries, references, and appendices, shall not exceed 80 pages for any thesis. .

Table 1.1: Chapter length estimates

Chapter	Min–Max pages
Introduction	3–5
Related Work	3–7
Method	6–15
Results and Analysis	7–20
Discussion	7–15
Conclusions and Future Work	4–8
TOTAL	30–70

DRAFT

Hacking products and services intended for malicious use are readily available on the darkweb marketplaces [36]. Such products and services are commonly used by cyber criminals. For example, the banking trojan Dyre whose goal was to steal credit card information, used an exploit that had been available on the darkweb for four months [36]. This suggests a lower barrier of entry, where the required technical skill of cyber criminals has been reduced.

Kaspersky reported that they detected on average over 467,000 malware daily during 2024. An increase of 14% compared to 2023 [29]. The AV Test Institute echoes a similar figure, 450,000 new malware daily [2]. Furthermore, they report that malware targeting Windows has increased by more than 420 million between 2020 and 2025, representing an increase of 74% [2].

In the report Cost of a Data Breach Report 2022, IBM reports that the average cost of a data breach is \$4.35 million. Furthermore, IBM reports that 11% of the breaches were due to ransomware attacks. An increase of 41% from the year before [27].

Sharmeen et al. argues that the availability of Ransomware-as-a-Service (RaaS) is one of the factors why ransomware attacks have increased [58], and that RaaS makes it possible for novices to make their own ransomware. The cost to the victim(s) of these attacks can be severe. For example, FedEx suffered \$300 million in losses due to a ransomware attack [58]. These observations demonstrate the need for continued malware analysis.

2.1 Malware Analysis

Malware analysis makes use of automation, but as a field it does not contain a fully automated pipeline. As a consequence, the field requires manual labor by professional analysts [65]. There are two methods of analysis, static and dynamic analysis.

Static analysis examines the suspicious file without executing it. This ranges from simple methods such as inspecting imported DLLs and stored string literals to more advanced techniques like reverse engineering. If reverse engineered, the file is disassembled, which allows the analyst to inspect the program's assembly code [59].

Such techniques make it possible to produce simple signatures [59]. Signatures can be used to confirm the presence of malware by scanning for byte patterns in a suspicious file and comparing it to known signatures [21]. However, some of those techniques are mostly ineffective against advanced malware [59].

Dynamic analysis, on the other hand, aims to execute the suspicious file in a controlled environment. This makes it possible to observe the program's behaviour. It enables the analyst to review, e.g., changes to the Windows registry or recorded Windows events, or stepping through the code in a debugger [59].

One type of tool that sees use during dynamic analysis are sandboxes. Sandboxes work by automatically launching a virtual machine (VM) and simulate network services. The malware is then executed inside the VM. During execution, the sandbox collects information such as registry and network activity. This information is then presented once the execution is complete [59].

Dynamic analysis is not as affected by static evasion techniques, such as obfuscation. However, it is not immune to evasive malware as a whole. There exist anti-analysis techniques that malware commonly employ in order to evade dynamic analysis [13, 42]. Sandboxes may fail to execute the malware if it expects a specific type of input [21]. For example, command line arguments [59] or a specific environment [21]. A malware may also fingerprint the environment in which it runs [17]. The privileged instruction `IN` in x86 can reveal a distinct artifact. Hypervisors may execute this instruction in user-mode, revealing hypervisor specific magic values [17]. If malware suspects that it runs in a sandbox it may simply choose not to execute its payload [13, 17, 42, 48].

Both methods have different pros and cons and thus complement each other. Identifying malware with a signature is faster, but requires that the malware is previously known. New signatures must be created for any new derivatives [21]. Sandboxes that relies on observing a malware's behaviour is time-consuming, because it has to create and launch a VM and then wait for the malware to finish execution [21].

For production environments, static analysis is still a needed alternative due to its speed and practicality [16]. Due to the fact that the malware is never executed, the analysis is both faster and require less computational resources than executing the malware in a sandbox [16]. Another benefit of static analysis is the fact that it has the capability to explore all possible execution paths of the extracted CFG. Limited only by path explosion. Whereas dynamic analysis may only show a small fraction if the malware managed to identify the analysis environment [16, 67], which then may require manual interaction [37].

2.1.1 Machine Learning

In recent years, the belief in cyber security is that Machine Learning (ML) and Deep Learning (DL) will improve malware detection [20]. Machine learning is good at processing a lot of data and can be used as an initial screening on suspicious files [21], and a recent systematic literature review shows that models that have been trained on known malware have the capability to detect unknown samples (i.e. samples not in the training data) [16].

The most popular end goal of malware analysis is to determine whether a file is malicious or not. However, there are other end goals [61]. Determining the maliciousness of a file remains the most important, because it allows prevention by choosing to block execution before harmful actions are taken. [61].

Finding similarities between malware samples is another such goal. This makes

it possible to understand how a novel sample is different from previously known samples. There are different variations of this goal [61]

- *Variants Detection*
- *Families Detection*
- *Similarities Detection*
- *Difference Detection*

An effective way for malware authors to avoid detection is by creating derivatives by reusing source code and resources. Variant detection aims to stop that strategy by attempting to detect whether a sample is such a derivative or not. In addition, it also help in the understanding of how a malware sample evolves over time [61].

Family detection aims to classify malware samples by family membership. This is achieved by comparing the sample to a selection of families from a known dataset [61].

Similarity detection works by finding which parts of a sample are the same or similar to the samples that have already been analyzed. This allows analysts to focus their analysis on the parts that are new and ignore the parts that were analyzed before [61].

Difference detection works the same as similarities detection. But, as the name suggests, it instead identifies differences. Being able to pin point differences can aid in the discovery of new functionality that requires more detailed analysis [61].

Machine learning models are used in static and dynamic analysis, or both together [20,61]. In the context of machine learning, the type of analysis is mainly about how you extract features [20], and which features are available [16].

However, the training of these models is not easy. In the end, the models effectiveness is highly dependent on the selected features [16], but also the quality of the dataset. Furthermore, malware classification can degrade by applying code transformations at the assembler stage as shown by Gibert *et al.* [21].

2.1.1.1 Static Analysis Features

A long list of feature classes can be extracted directly from the binary file. Windows binary files (.exe and .dll) are known as Portable Executables (PE), and their file header contains metadata such as imported third party libraries and functions, where the file stores the machine code, when it was compiled, etc. Strings are another type of feature class that exists inside a PE file and is commonly used [16]. In addition, bytes and/or byte sequences can be gathered directly from the file's content [11,37,67].

A disassembler is an external tool that can be used to disassemble a binary file. This is when the disassembler attempts to translate a binary's machine code back into human readable assembly. A single machine operation is known as an opcode. IDA Pro is an example of a disassembler and makes it possible to extract additional feature classes such as opcode sequences and Control Flow Graphs (CFG) [11,62].

The feature class most commonly used by machine learning is n -grams [20,61]. An n -gram is a contiguous range of n elements [20,61] taken from a vector. In the context of malware analysis, this is usually bytes taken from the malware file or opcodes from the disassembled output [20]. For the latter, n -grams are created by extracting every n consecutive opcode. These opcodes operate on parameters known

as operands, and they are usually discarded while the assembly name of the opcode is kept, i.e. `LEA`, `PUSH`, `MOV`, etc [20]. Such names are also known as mnemonics.

There exists a potential risk with byte n -grams which is that their learning appears to come mainly from strings. Specifically, strings that exist inside the PE header [49]. Meanwhile, opcode based approaches are more robust against certain obfuscation techniques [21].

The size of n is important as it results in a tradeoff between prediction accuracy and computation cost. The number of features increases exponentially with n [61]. Trigrams ($n = 3$) are a good middle-ground between computational complexity and accuracy. Most papers use $n \leq 3$ [61].

In the study conducted by Gibert *et al.* they extracted 9,514,156 unique byte 3-grams from their dataset. Compared to 55,136 unique opcode 3-grams [21]. The large amount of instructions available in x86-64 appears to not drive n -gram growth. This phenomenon could possibly be explained by the fact that C/C++ programs tend to use a small and skewed subset of all available instructions [3]. In that sense, assembly language acts similarly to an ordinary language. A small number of instructions are used over and over, while many others are relatively rare. Specifically, the frequency of instruction can drop according to a power law distribution [14,15].

2.1.2 Evasive Malware

Code obfuscation is one of the more common evasion techniques used by real world malware [18]. It works by making changes to the structure of the malware's code [9,18]. It may also make changes to the malware's control flow and should preserve the functionality of the unaltered code [18]. Control flow flattening and dead code insertion are two examples of such techniques. The former introduces noise, making it more difficult to reconstruct the real path through the program. For example, a programs `if` statements may be replaced by a single large `switch` statement [56] and may have additional bogus cases in it [18]. The latter adds assembly code that does nothing or cancels itself out, such as adding a value to a variable and then immediately afterwards subtracting that same value [18].

The creation of a binary happens in stages. In the context of obfuscation, the interesting stages are *compilation*, *assembly*, and *linking*. Compilation operates on high level source code which can be obfuscated with tools such as Tigress [63]. The compiler translates that high level source code into assembly code. That produced assembly code can itself have obfuscation added to it [21] before being passed on to the assembler. The assembler turns the assembly code into machine code that is stored in `.obj` files. Finally, the linker links the produced `.obj` files into a single binary. The final binary can then be further obfuscated with the help of packers that compress it [59]. Obfuscation can therefore be done in layers and can be used in combination because they are not mutually exclusive.

2.2 Problem Statement

Each stage in the build process operates on the input created by the previous stage. Changes in one stage therefore have a cascading effect throughout the build process.

Different high level source code produce different assembly code. Different assembly code produce different `.obj` files. Different `.obj` files produce different binaries. Obfuscation techniques therefore change how the final binary looks like [61], regardless of which layer it is applied. As the binary changes, so do its static features. This affects the features that can be extracted from it [16].

The assembly code produced by the compiler is not solely dependent on the high level source code on which it operates. The compiler exposes multiple settings known as compiler flags. These allow the programmer to influence how the compiler generates assembly code [39]. The same high level source code can therefore produce different assembly code, and thus impact the composition of final binary.

The translation of high level source code into machine code is therefore not unique and the outcome depends on compilation choices (e.g. optimization level, hardening flags, etc). All of which can alter instruction selection, order, magnitude, etc. This matters because ML models trained for static malware detection and classification depend on features extracted from binaries, such as opcode n -grams, which are often frequency based [21, 61]. This means that as any technique makes changes to the underlying instruction order and frequency, the extracted opcode distribution will change. The thesis will refer to such changes as feature drift.

If different compiler configurations can induce feature drift similar to obfuscation techniques, then extracted opcode n -grams features can experience distribution drift from the compiler alone. Such drift matters because a model trained on opcode n -grams is at risk of learning patterns related to specific compilation settings that exist within the training data, instead of patterns related to the program logic. The same program recompiled with different compiler flags will have its functionality preserved but may have a different opcode distribution. As a result, the model prediction accuracy may be worse because the feature distribution is outside the training data, and not because the program has different functionality. This reduces a model's ability to generalize and raises an important dataset problem. If binary samples in a dataset were produced by an undocumented or limited set of compilation settings, then the dataset may not properly represent the variability that can exist in real world binaries.

In many studies, extracted n -gram feature vectors are implicitly treated as a stable representation of the functionality of the program. However, if the compiler settings can impact that vector, then the vector is not dictated by program behaviour alone but also reflects a controllable build configuration. If true, then such a configuration cannot be treated as an implementation detail and should instead be treated as a controlled experimental variable that can impact the extracted features. Dataset construction and robustness evaluations should therefore report and try to diversify such configurations.

2.2.1 Aim

The goal of this thesis is to quantify compiler induced feature drift in opcode n -gram distributions of size 3. The study conducted will measure the divergence between program specific baseline binaries and binaries recompiled with a set of MSVC flag configurations. The measured compiler induced drift is then compared to drift induced by source code obfuscation techniques from Tigress. The purpose of the study

is not to evaluate a specific classifier, but to study whether compilation choices can produce feature drift of similar magnitude as an established source code obfuscation tool. Thus, the study aims to determine whether build configurations can produce enough feature drift to matter for robustness evaluation and dataset construction.

2.2.1.1 Scope

The study targets Windows and x86-64, a single compiler (MSVC), a subset of available compiler flags, and a fixed set of source code transformations. Seven custom C modules were created to mimic malware behaviour. These seven modules were combined using pre-processor directives to create 21 different programs.

Disassembling and opcode extraction was done with the help of IDA Pro. Once the n -gram distributions had been created, the divergence were computed using Jensen-Shannon Divergence (JSD).

End-to-end detector training or evasion experiments and multiple feature classes (such as CFG or API-calls) are both out of scope. Instead, potential evasion is inferred from related work.

2.3 Thesis Structure

TODO: This will be done when the complete structure is finalized.

3.1 Feature Drift & Malware Evasion

Machine learning techniques in malware analysis assume that training data and testing data follow an identical underlying distribution [10,33]. Breaking this assumption is what enabled evasive attacks, argued by Chen *et al.* [10] and speculated by Li *et al.* [33]. Malware authors are incentivized to apply adversarial attacks [20]. The use of adversarial attacks to trick classifiers has been demonstrated by multiple studies [4, 10, 35].

3.1.1 Evasive Malware

Chen *et al.* showed that targeted feature manipulation can lead to accuracy degradation in classifiers. The authors fooled the model by adding and removing Win32 API calls. The model had been trained to view some API calls as more linked to benign intent, while others were more linked to malicious intent. The authors exploited this by increasing the number of benign API calls and reducing malicious ones, which enabled them to create evasive samples [10].

Al-Dujaili *et al.* achieved a very high evasion rate against a malware classifier that had been trained without adversarial samples. They achieved evasion rates of up to 99.7%, by adding data in feature space. Their adversarial samples had data added to the binaries while keeping the original data intact. In essence, the authors created a superset of the malware’s original features. They managed to reduce the evasion rate to as low as 5.9% for some classifiers by augmenting the training data with adversarial samples. [4].

The previous two studies focused on API calls, which is a different feature class than opcode n -grams. Their work does not study feature drift induced as a result of an evasion technique, but it shows that targeted changes in feature space can degrade malware classifiers. In addition, the results from Al-Dujaili *et al.* emphasize that robustness in classifiers depends on whether feature drift was taken into account during training. This motivates quantifying feature drift, which can help validate feature diversity of the training data.

3.1.2 Code Transformation

Gibert *et al.* conducted an audit of static ML detectors to test their robustness against code obfuscation. Some of the models tested relied on opcode n -grams as

a feature class. The obfuscation techniques they used included dead-code insertion, register reassignment, subroutine reordering, and code reordering through jumps [21].

They found that opcodes outperformed byte-based approaches and that byte-based approaches were more susceptible to register reassignment. However, they also managed to fool some opcode-based classifiers. This was accomplished with the help of the obfuscation technique dead-code insertion. For their study, this technique inserted multiple NOP instructions in the disassembled malware code [21]. These instructions do not do anything (functionality preserving), but make changes in feature space that affect the extracted n -gram features.

Gibert *et al.* argues that the addition of NOP instructions introduced a data-bias issue. The reason was that some of the malware families that the classifier trained on lacked NOP instructions. They managed to improve robustness by augmenting the dataset [21]. Their work is directly related to the work in this thesis, as it provides evidence that functionality preserving obfuscation can affect opcode n -grams in such a way that it degrades classifier performance. In addition, it shows that such degradation can to some extent be counter-acted by having adversarial samples in the training data.

Gibert *et al.* showed that dead code insertion can change a malware enough to fool a classifier [21]. If making changes to the underlying `.text` section to change the opcode n -gram distribution enough to fool static detectors. This suggests that the compiler itself is capable of this. Evidence presented by Ren *et al.* indicate that this is the case [51].

3.1.2.1 Compiler Induced Transformations

Ren *et al.* used a genetic algorithm whose purpose was to find the set of non-default compiler optimization flags that would result in a large code difference. Their primary research focus was binary diffing and not malware analysis. Because of this, they focused on concepts such as CFG and basic blocks instead of n -grams which has a tendency to produce false negatives in binary diffing. However, they did briefly explore the use of their algorithm as a malware evasion technique [51].

They applied the algorithm to LightAidra, an IRC botnet whose source code is available online. The binary produced was then uploaded to VirusTotal. The transformed binary resulted in a reduction in engine recognition by more than 50%, compared to a LightAidra binary produced by compiling with `-O3`. Ren *et al.* reached the conclusion that if a non-default optimization flag space is searched, then the produced binary can differ substantially [51].

The authors also collected data of a year's worth of Mirai binaries, a popular IoT botnet. The data showed that up to 42% of Mirai variants were compiled using non-default GCC optimizations. In addition, the data also showed that these variants exhibited a lower detection rate compared to the other samples [51].

Their work is highly relevant, as it provides evidence that compiling real malware source code with a set of carefully selected compiler flags makes it possible to evade detection in live malware detection systems. Their work also shows that the compiler has been used as a tool to create evasive Mirai variants in the wild.

Ren *et al.* continue their work on compiler optimizations in the context of binary diffing in a later paper [52]. This time, the authors looked at optimizations applied

by the compiler to local code. A technique known as peephole optimization. The compiler achieves this by rewriting instructions with alternatives that are either more compact, more performant, or both. Ren *et al.* monitored and measured the effect that such optimizations had on basic blocks. Their results showed that peephole optimizations affected between 32.8% and 60.2% of all basic blocks on average. In addition, their work shows that the impact of compiler optimizations on the code is not isolated to local instructions, but can introduce transformations that alter the structure of basic-blocks and the shape of the CFG [52].

Their findings provide evidence that semantic-preserving rewrites done by the compiler happen sufficiently frequently to plausibly affect static-analysis features. The altered structure of basic-blocks and re-shaped CFG is highly relevant to feature extraction. Because when optimizations merge or split blocks, remove branches, or replace blocks with specialized instructions, any feature that relies on local instructions being stable can change drastically even though a programs functionality remains the same. The work done by Ren *et al.* motivates the study of feature drift in malware detection, which also relies on static features related to code.

3.1.3 Source Code Obfuscation

In a recent survey, Geng *et al.* emphasizes that malware authors still prioritize transformation-based evasion, such as code obfuscation. Malware authors may take advantage of tools that apply code obfuscation at the compilation stage [18]. Malware that has been obfuscated changes its signature but preserves its functionality [9]. Geng *et al.* categorize code obfuscation as either data-based or control flow-based. Control flow flattening aims to split a program’s control flow with the help of random jumps, junk code, or fake jumps, resulting in different basic blocks [18]. Chandran *et al.* further add subroutine rearrangement and instruction replacement to the list of obfuscation techniques [9]. Real world malware repeatedly uses multiple obfuscation techniques at a time [9, 18].

Data-based obfuscation changes the way the data of a program are stored. Such techniques typically also make changes to the code because the original data have to be restored. For example, variables can be split into multiple smaller components. The original value is later reconstructed at runtime with, e.g. XOR operations [56].

The work in the previous two paragraphs provides an overview of which types of code obfuscations are used by malware. This provides motivation for using Tigress as a controlled proxy of adding obfuscations, because Tigress can apply the same type of transformations emphasized in the surveys, by making mutations to control flow, data and code layout. Furthermore, it aligns with the statement from Geng *et al.*, that malware authors can use tools to apply obfuscation at the compilation and linking stage.

3.2 Malware Analysis Benchmarking Issues

Ucci *et al.* note that there is no widely available reference benchmark in malware analysis research that can be used to compare results across studies [62]. There appears to be an exception to this for Windows PE files. That is, Microsoft’s BIG

2015 dataset which is hosted on Kaggle [20, 53]. However, Gibert *et al.* note that this benchmark has limitations because it contains files as hexadecimal data without their PE headers, together with the disassembled output of those files. This makes it impossible to replicate the disassembly process [20]. Furthermore, Ucci *et al.* also emphasize that the datasets malware analysis research uses for the evaluation are not always made public. As a result, it is difficult to compare the reported performance across papers reliably [62]. The same concern is raised by Gibert *et al.* They point out that accuracy figures across studies are not directly comparable if authors use distinct datasets, since those datasets may have different labeling practices [20]. Meanwhile, Maiorca *et al.* further note that many papers that study PDF malware detection do not report the exact sample composition of their dataset [35].

One of the reasons for the lack of a widely available public benchmark is for legal and safety reasons [5, 20, 66]. Models trained for malware analysis typically require that the dataset it is trained on contains both malware and legitimate software [5, 20]. The binaries of legitimate software cannot normally be shared because they are protected by copyright [5, 20, 66]. In addition, sharing real malicious files comes with security concerns [5, 66], but they can also suffer from restrictive licensing that prevent sharing. For example, VirusTotal provides data that can be used internally, but disallows sharing of that data to other parties [5, 20]. These constraints impact what researchers can publish and restrict the reproducibility of robustness evaluations.

Datasets such as EMBER attempt to solve these issues by shipping features instead of binaries [5]. Other datasets, such as SOREL-20M, ships disarmed (i.e. modified) malware files in addition to features [26]. The copyright issue is solved by shipping only features instead of intact binaries. However, this makes it difficult to extract new distinct features [16]. A likely consequence of feature-only datasets is that they may prevent the generation of paired "original and transformed" binaries. This in turn makes it difficult to test how a trained model would react to adversarial samples that were created as transformed derivatives of malware in its training data.

3.2.1 Implications

These dataset realities impact the methodology options that are available. As a consequence, much of the literature either operates on binaries directly or on their disassembled counter-parts [4, 10, 11, 21, 30, 37]. For example, the robustness audit conducted by Gibert *et al.* applied its code obfuscation at the assembly stage [21]. The highest stage possible with BIG 2015 which does not contain high level source code.

This study has opted to focus on quantifying the actual feature drift instead of proving evasion. The reason is that the lack of standardized benchmarks limits the comparability and external validity of end-to-end evasion results. Feature drift is instead computed on opcode n -grams using Jensen Shannon Divergence (JSD) as a model agnostic metric that measures distributional change. However, such experiment requires access to high level source code and controlling the compiler variable.

Without access to the high level source code (such as C or C++), it makes it impossible to study compiler transformations under controlled conditions. High level source code from real world malware does exist, such as TinyNuke [25], SilverRAT

[24], Mirai [23], Babuk Ransomware [22] or LightAidra that Ren *et al.* used in their study [51]. However, collecting them all into a single dataset is not straightforward because leaked malware usually uses different build tools. Babuk uses MSBuild, specifically Visual Studio 2019 [22], Mirai is written for GCC [23] and TinyNuke relies on Visual Studio 2012 [25]. The consequence of this is that the compiler version cannot be a controlled variable.

3.2.1.1 Alternative Solution

Creating custom C modules that mimic malware behaviour is an option that allows controlling the compiler variable during the experiment. This makes it possible to isolate the effects of compiler configurations during repeated recompilation. Another benefit is that the source code can be publicly shared because it avoids third-party copyright restrictions. This work therefore uses a custom set of C modules that implement common MITRE ATT&CK techniques in a non-harmful manner.

Self-authored code like this has its limitations. It sees a fair amount of use in Software Protection research and has received criticism by De Sutter *et al.* in their survey "*Evaluation Methodologies in Software Protection Research*". The authors use the term "toy program", which they define as tiny and basic programs with a special purpose, implementations of well established algorithms (such as bubble sort or binary search), or synthetic programs that are the result of some tool (such as Tigress) [12].

De Sutter *et al.* argues that such toy programs can be a threat to external validity due to not representing the true complexity of real world software, but notes that it does not have to be. For example, if the protections can be evaluated locally without depending on the full program. They further argue that toy programs must be made available so as not to threaten reproducibility [12].

3.3 Research Gap

Previous work shows that the performance of machine learning models that rely on static features can be degraded by code transformations and targeted feature manipulation. In addition, such actions preserve the harmful functionality of the transformed malware. Specifically, studies on code obfuscation and evasive malware show that mutations to the binary, such as control-flow manipulation or changes to the program structure, can cause feature representations to drift and reduce a model's accuracy as a consequence. This motivates the need for robustness testing and emphasizes that binaries which are functionally equivalent may still map to distinct feature vectors.

Research also shows that compilation choices can affect binary similarity and, in some cases, malware detection outcomes. Optimization levels and code generation settings may change the produced instruction sequences, control-flow structure, basic blocks, etc. However, while obfuscation and adversarial samples are often mentioned and addressed in the literature, compiler configurations are often missing as a variable. Instead of being treated as a core driver of feature drift and studied.

There is limited quantification of feature drift in opcode n -grams in the reviewed

literature. Specifically, drift induced by compiler flag combinations in Windows PE files. In addition, many studies evaluate obfuscation or adversarial effects on detection models. However, not many papers directly compare those to compiler induced equivalents. Work that attempts to use a model agnostic drift metric is rare, despite the difficulties in malware analysis to cross compare results due to the lack of public benchmarks.

This work will address those gaps by conducting a controlled, model agnostic experiment of opcode 3-gram distribution drift induced by MSVC flag configurations. That drift will be compared with drift induced by traditional code obfuscation techniques for valuable context. By keeping higher level source code constant and switching compiler configurations, extracting 3-grams, and measuring using Jensen-Shannon divergence in a structured manner. The thesis will provide empirical evidence on the drift magnitude caused by a subset of compiler flags available in MSVC and show how it compares to obfuscation induced drift. This evidence will help future robustness evaluation and dataset construction methods by making it possible to quantify distribution diversity.

4.1 Research Questions

- **RQ1:** To what extent can opcode 3-grams drift by the tested compiler flags?
- **RQ2:** To what extent can opcode 3-grams drift by the tested Tigress transformation set?
- **RQ3:** To what extent do the tested compiler flag configurations produce opcode 3-gram drift comparable to the tested Tigress transformation set?
- **RQ4:** Which of the included compiler flags is associated with the largest mean opcode 3-gram drift?

Previous work suggests that transformations can change static feature representation and degrade detectors, but compiler configurations are often not taken into account in such studies. RQ1 therefore quantifies the magnitude of opcode 3-gram drift induced by a set of tested compiler configurations. Hence, addressing the lack of quantified compiler-induced 3-gram drift in Windows PE binaries. RQ2 quantifies the drift induced by Tigress transformations, so that compiler-induced drift can be compared against established code obfuscation techniques that can be observed in real world malware.

Previous work lacks a direct comparison between compiler-induced and obfuscation-induced drift, although both are plausible sources of such drift. By comparing compiler-induced and Tigress-induced drift, RQ3 helps answer whether tested compiler configurations can create drift that rivals established code obfuscation techniques.

Calculating overall drift does not in itself provide information about which compiler flags are most responsible for any created drift. RQ4 therefore estimates which individual flags cause the largest mean drift. In turn, providing a more interpretable result over which compiler flags impact opcode 3-gram drift the most.

4.2 Experiment Design

This study uses a comparative experiment under controlled conditions that follows a repeated measures design. Its purpose is to quantify how much opcode 3-gram distributions from the same program can drift when it is affected by different compilation

settings or code obfuscation. A repeated measures design is appropriate because the research questions focus on feature drift caused by code generation, and not differences between programs. This design makes it possible to keep the program constant while observing it under code obfuscation and several distinct compiler flags.

Any design that relies on comparing program-to-program across compiler flags is not appropriate because it would confound compilation drift with program differences. The reason an end-to-end detector experiment was not chosen as a methodology was because the goal of the thesis was to quantify feature drift under controlled conditions and not to evaluate a specific classifier. Datasets, feature pipelines, and model choices can all affect the results of transformations in an end-to-end detector design. The lack of a standardized and reproducible benchmark means that such a design does not guarantee better external validity, since cross classifier comparability is limited. The used design therefore quantifies the drift of 3-gram distributions in a model agnostic way, while potential evasion is inferred from related work.

4.2.1 Design Overview

The generated programs serve as experimental units of the study. Each program was first compiled with /O2 to serve as the baseline condition of that program. The use of optimized baselines was motivated by the recommendation from De Sutter *et al.* to avoid unoptimized binaries in software protection research [12]. The program was then recompiled with each of the compiler flag configurations. Finally, the program was obfuscated using Tigress and compiled with the baseline configuration as the final and separate stage. Then each binary produced by a treatment was compared to its baseline variant.

The compiler flag configuration and the Tigress transformation are the independent variables. The dependent variable is the JSD between two opcode 3-gram distributions collected from the transformed binary and its baseline variant. Source code, software and tooling versions, process architecture, and data-collection are treated as controlled variables. This makes it possible to map the observed distribution divergence primarily to the applied configuration instead of differences in implementation or data collection.

Control Variables	Independent Variable	Dependent Variable
Software versions	Compiler flags	Jensen-Shannon Divergence
Source code	Tigress obfuscation	
Data collection		
Architecture		

Table 4.1: Variables

This design is a model agnostic way to study feature drift under controlled conditions caused by compiler flags, and can answer all four research questions. RQ1 and RQ2 are answered by estimating the drift magnitude between the baseline and compiler and Tigress transformed binaries, respectively. RQ3 is answered by comparing compiler induced and Tigress induced drift with the help of the signed mean difference between their mean JSD values. In addition, RQ3 also answers the probability

that a randomly selected Tigress induced drift value exceeds a randomly chosen compiler induced drift value for a particular program. RQ4 is answered by modeling the contribution of individual compiler flags while taking into account program variation with the help of a mixed effects analysis.

4.3 C Modules

Rahman et al. should maybe be in related work as well.

In total, there are seven unique C modules that implement techniques from MITRE ATT&CK. The techniques chosen are a subset of the high frequency techniques presented in the survey from Rahman *et al.* [50]. Their survey collected public cyberthreat intelligence reports (CTI) on advanced persistent threat (APT) attacks. From these reports, the authors extracted information about which MITRE ATT&CK techniques that were commonly used. The techniques used in this thesis are the following.

- T1082, System information discovery (Discovery)
- T1083, File and directory discovery (Discovery)
- T1057 Process discovery (Discovery)
- T1070.004, File Deletion (Defense-evasion)
- T1547.001, Registry run keys/ Startup folder (Persistence)
- T1059.001, PowerShell (Execution)
- T1005, Data from local system (Collection)

All modules resolve their respective Win32 API calls through runtime linking. Table 4.2 contains short descriptions of the implemented modules and whether or not they log their output. This is done as an alternative to exfiltrating data over the internet. The log is placed on the `Desktop` for the current user in a directory called `output`. The filename of the log contains the name of the ATT&CK technique and a randomly generated hex value as its suffix. The complete source code for all modules and the corresponding CMake script are available on GitHub [1].

4.3.1 Compiled Programs

The different modules were combined into three at compile time to make up a final program. Compile time modularity was achieved with the pre-processor. Each module implementation was placed behind a pre-processor directive, named after the MITRE technique that module implements.

The choice was made not to evaluate the full combination space of $\binom{7}{3} = 35$ programs. Instead, a balanced incomplete block design (BIBD) with 21 programs was used. This helps reduce the number of programs, without sacrificing balanced representation of individual modules and module pairs.

Module	Description	Logs	Note
T1005	Scans the file system for files matching a specific file extension.	✓	Full list of file extensions can be seen in Appendix A.
T1057	Enumerates the processes currently running. Exits if any of them are part of a blacklist of known virtual machine, anti virus, etc. Otherwise collects the processes privilege rights.	✓	–
T1082	Collects system information related to hardware, networking and the operating system.	✓	–
T1083	Scans the file system for unsigned DLL files. Checks for both embedded certificates and the Windows catalog.	✓	–
T1059.001	Executes a PowerShell script three times that echos " <i>Hello PowerShell!</i> ".	✗	The scripts are executed by calling <code>CreateProcessW</code> , <code>ShellExecuteW</code> and <code>WinExec</code> once per script.
T1070.004	Erases the logs and the program binary from the file system.	✗	
T1547.001	Adds a new Run key entry and a link file to the program in the Startup folder.	✗	Defaults to <code>HKEY_LOCAL_MACHINE</code> but falls back to <code>HKEY_CURRENT_USER</code> if it lacks privileges.

Table 4.2: Description of the implemented C modules and whether or not they log their output.

4.3.1.1 Balanced Incomplete Block Design

In the context of the work in this thesis, each block is a program, each treatment is a module, and the block size is thus the number of modules per program. Let b be the number of programs, g be the number of modules, and k be the number of modules per program. A BIBD exists *iff* the two balancing conditions are met [68].

1. Each module must not appear more than once per program and all modules must have the same number of replicates (r) [68].
2. Each pair of modules must appear in a program the same number of times (λ) [68].

The first condition can be checked by computing r using $N = bk = gr$, where N is the number of experimental units and r is the number of replicates per module [34, 60, 68].

$$N = bk = gr \implies 63 = 21 \cdot 3 = 7r \implies r = \frac{63}{7} = 9$$

The second condition can be checked by computing λ from the equation $r(k-1) = \lambda(g-1)$ [34, 60, 68].

$$r(k-1) = \lambda(g-1) \implies \lambda = \frac{r(k-1)}{(g-1)} = \frac{9(3-1)}{(7-1)} = \frac{18}{6} = 3$$

Table 4.3: Mapping of modules to programs that fulfills $r = 9$ and $\lambda = 3$.

Module	Program																				
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
T1082	✓	✓	✓	✓	✓	✓	✓	✓	✓												
T1083	✓	✓	✓							✓	✓	✓	✓	✓	✓						
T1057	✓			✓	✓					✓	✓					✓	✓	✓	✓		
T1070.004		✓				✓	✓					✓	✓			✓	✓	✓		✓	
T1547.001			✓			✓		✓		✓				✓		✓			✓	✓	✓
T1059.001					✓		✓		✓			✓		✓	✓		✓		✓		✓
T1005				✓				✓	✓		✓		✓		✓			✓		✓	✓

4.4 Compiler Flags

There are more than 30 compiler flags listed under "Code generation" in Microsoft's documentation [39]. Some of the flags listed have binary state (on/off), others have multiple levels (such as `/arch:SSE2`, `/arch:AVX` and `/arch:AVX2`). Some flags have been deprecated (e.g. `/Ge`), and others are mutually exclusive (e.g. `/RTC1` and `/O2`).

Testing all available flags on 21 programs quickly becomes unmanageable, because the number of produced binaries increases exponentially. For flags that have a binary state, the number of binaries produced can be calculated with $n_b = 21 \cdot 2^{n_f}$. Six flags ($n_f = 6$) result in $n_b = 1344$, for seven flags this becomes $n_b = 2688$, and for eight flags it becomes $n_b = 5376$. The implication of this is that only a small subset of the available flags can reasonably be tested.

The largest n_b considered manageable was 1344, which is $n_f = 6$. However, a fractional factorial set was used instead of a full factorial set because it makes it possible to increase the size of the final subset to 8 flags and still reason about pairwise interactions (e.g. $F_1 F_2$) without ambiguity. Specifically, 64 runs with Resolution V is possible with 8 factors and have two factor interactions only alias with three factor (and higher) [47].

4.4.1 Excluded Flags

The optimization flag `/O2` is a combination of multiple flags. Specifically, `/Og /Oi /Ot /Ob2 /GF /Gy`. Because `/O2` is used as the baseline, these flags will not be in the final subset of flags tested. There are multiple flags that are only available for x86 and not x86-64. That is, `/Oy /Gd /Gr /Gz /QIfist /Qsafe_fp_loads /hotpatch`. To keep the number of measurements under control, only 64 bit binaries will be compiled. `/RTC1` (and all its variants) is another flag that can be excluded, since it is incompatible with optimized builds (`/O2`).

The flag `/jumpablerdata` affects switch statements that have been optimized into jump tables. It is responsible for moving the jump table out of the `.text` section and into the `.rdata` section. This flag is excluded because none of the modules contains any switch statements. The `/GT` flag affects variables declared with the keyword `__declspec(thread)`, and because that keyword is never used this flag is excluded.

`/Qpar` is MSVC's auto-parallelization flag. It requires that loops have been prefixed with `#pragma loop(hint_parallel(n))`. This flag can add or remove quite a lot of assembly code. However, auto-parallelization can't be applied to every loop as it depends on whether or not the loop has any data dependencies and if the compiler can prove that it can safely be parallelized.

The flags `/fp:contract /fp:except /fp:fast /fp:precise /fp:strict` are all related to floating point arithmetic. These 5 flags are excluded because they only affect floating point arithmetic, which is not the primary goal of any of the modules.

The flag `/feature` is excluded because it is only available for ARM64. The security flag `/guard:ehcont` generates a list of relative virtual addresses that are valid for exception handling. This is only metadata and therefore it would typically only affect byte n-grams and not opcode n-grams.

Processor extensions related to SIMD can be enabled or disabled with the flag `/arch:[option]`. Multiple options are available, such as `SSE2`, `SSE4.2`, `AVX`, `AVX2`, etc. These extensions can introduce new instructions to x86-64, for example `vmovsd` from `AVX`. These instructions operate on the vector registers (`XMM`, `YMM`, etc.) and are therefore related to floating-point variables and SIMD operations. This flag is excluded because floating point and SIMD arithmetic is not a primary goal of any of the modules.

The last flag to be excluded is `/fsanitize=address`. It is responsible for enabling Microsoft's address sanitizer. It inserts instrumentation and can create large changes to the produced assembly code. However, it is excluded because it statically links a lot of code in the produce binary which risks confound the controlled source code with instrumentation code.

4.4.2 Included Flags

The `/GL` flag enables optimizations for the whole program. Concretely, it can result in optimizations such as reducing the amount of loads and stores and inline functions across modules. This is included because the source code consists of multiple C modules and it is thus likely to benefit from such optimizations.

The three flags `/Qspectre`, `/Qspectre-load` and `/Qspectre-load-cf` are responsible for mitigating the Spectre vulnerability. `/Qspectre-load` is included because it is the most intrusive among the three, forcing the compiler to insert code at all possible locations where the vulnerability may occur. See Appendix B.2. This flag is included because the modules make extensive use of memory loading operations.

The calling convention in Microsoft's 64 bit Application Binary Interface (ABI) uses four registers to pass the first four parameters. In addition, this calling convention always allocates 32 bytes on the stack where the callee can optionally store the passed parameters. The flag `/homeparams` make it possible to enforce the storage of parameters, see Appendix B.3. This flag is included because it impacts the generated code of every function that accepts parameters.

The flag `/guard:cf` generates security checks for indirect calls, such as function pointers. This flag is included because the implemented modules rely on runtime linking and thus calls Win32 through function pointers.

The flags `/Gh` and `/GH` are identical in nature. They both insert a `call` instruction to a user defined function in every compiled procedure, see Appendix B.4. Because they in effect do the same thing, only `/Gh` is included.

`/Gs[size]` is included because it makes it possible to insert stack probes into most functions. The optional size argument is the amount of stack allocated bytes that a function is allowed to use before requiring a stack probe. If the size is set to 0 a `call __chkstk` will be inserted into every function that uses stack allocated variables. Since this is not a binary flag, `/Gs0` will be used to represent "on", and its default value of 4096 will represent "off".

`/QIntel-jcc-erratum` is an optimization flag that strives to improve performance on certain Intel CPUs. It does this by inserting padding bytes (e.g. NOP padding) to align jump instructions to 32 bytes, if those jumps would otherwise cross that alignment. It is included because such padding can modify the final code layout and, as a consequence, may affect the extracted opcode 3-grams.

Code optimization for a specific architecture or micro-architecture can be controlled with the multi-option flag `/favor:[option]`. The flag is enabled by default and set to `/favor:blend` [40]. If this flag is set to `/favor:INTEL64` MSVC will attempt to optimize the code for Intel 64 bit processors. This flag is included to test whether such optimizations can affect the produced opcode 3-grams.

$$F_1 = \left\{ \begin{array}{l} \text{/homeparams, /Qspectre-load, /Gs0, /favor:INTEL64,} \\ \text{/Gh, /QIntel-jcc-erratum, /guard:cf, /GL} \end{array} \right\}$$

4.4.3 Flag Configurations

After creating the subset of eight included compiler flags, the final flag configurations were defined by a 64-run Resolution V fractional factorial design. Each of the eight

flags was treated as a two-level factor, each level representing an on- or off state. The multi-option flags were treated as a binary flag by using pre-determined options. These options can be seen in Table 4.4. In the design matrix created, each row represents a single flag configuration and all configurations were applied to every program. The matrix can be seen in Appendix A and may be referred to as a set of flag configurations F_2 , for the rest of the thesis.

Multi-option Flag	On State	Off State
/Gs[size]	/Gs0	/Gs4096 (default)
/favor:[option]	/favor:INTEL64	/favor:blend (default)

Table 4.4: Used multi-option flags and their respective on/off state when used as two factors.

4.5 Tigress Transformations

In total, four Tigress transformations were used. They were applied at the same time to all functions. The use of multiple transformations is motivated by the fact that real world malware often uses more than one type of obfuscation [9, 18]. Each program was transformed by Tigress in 16 repeated randomized runs and used the command-line option `--Seed=0`. In total, this produced $21 \cdot 16$ number of obfuscated binaries. Randomized runs were used because, in the Tigress transformation pipeline used, distinct fixed seeds produced identical `.text` section hashes, which `--Seed=0` did not.

$$R = \{r_1, r_2, \dots, r_{16}\}$$

$$T = \left\{ \begin{array}{l} \text{Flatten, Encode Literals,} \\ \text{RndArgs, Split} \end{array} \right\}$$

Flatten works by eliminating structured control flow, such as loops and `if` statements by replacing them with dispatch tables that can be built from, e.g., `goto` or `switch` statements [44]. **Flatten** was chosen because its end result is a reshaped CFG with different basic blocks. This matches the definition of control flow flattening as defined by Geng *et al.* [18]

Encode Literals works by obfuscating integer literals and is used in conjunction with the option `--EncodeLiteralsIntegerKinds=split`. This causes a literal integer to be replaced by many smaller components. These smaller components are then obfuscated by **EncodeArithmetic** [43]. This transformation was chosen because it attempts to conceal data. This makes it a type of data-based obfuscation which was discussed by Schrittwieser *et al.* [56] and Chandran *et al.* [9]. The strings were not encoded because the modules used wide strings, which the **EncodeLiterals** transformation does not support.

Split works by splitting functions into smaller functions [46]. It serves as a way to reshape function boundaries by targeting the program structure. The inclusion

of `Split` is motivated because Schrittwieser *et al.* names function splitting as an obfuscation technique [56].

`RndArgs` shuffles the order of the arguments passed into a function. The transformation was used in conjunction with the option `--RndArgsBogusNo=2`, which additionally causes it to insert two random arguments as noise [45]. This transformation is used because the instructions that pass the parameters may be reordered when the original arguments are shuffled. Furthermore, the bogus arguments can be seen as noise that closely resembles that of garbage code since it does not affect the programs final execution path/state. Both techniques are discussed by Schrittwieser *et al.* [56].

4.6 Compilation Pipeline

Compilation is automated by a Python script. For each flag combination, the script loops over each program and configures and builds the program using CMake. CMake is executed by using Python's `subprocess` library. To make sure that each program compiles cleanly with no trailing cache variables, the build directory is deleted before the configuration occurs. Build failures are logged in a file called `compile.log`. Only errors are logged, thus this file is expected to be empty if everything succeeds.

Compiler flags are set as environment variables which is then passed to `run` function that launches the process. This ensures that each process is launched with new environment variables. MSVC uses the variable `_CL_` to store compiler flags that should be appended [38]. Because these flags are appended, they will overwrite any conflicting compiler flags that have already been set.

If any compiler flag requires a linker flag to be set, they are set using the `_LINK_` variable. This variable works in a similar manner and appends the flags to the end. Guaranteeing that they are the dominant flags that the linker will use.

4.6.1 Tigress

The Tigress compilation script executes four key steps. The first step merges all of the C files required by the program into a single C file. This is required because Tigress expects a single source file as input. This is accomplished with the Tigress `--Merge` option.

The source file generated by the Tigress merge functionality may contain non-conformant C, such as `__declspec(...)`. This is an extension that is available for MSVC. The second step scans the generated file and removes such compiler extensions, because they may prevent Tigress from applying transformations and must therefore be removed.

Next, the script applies the transformations in T to the merged C file. The `Flatten` and `Split` transformations use their default options, while `RndArgs` and `EncodeLiterals` overwrite a subset of their default options, see Table 4.5.

The fourth and final step compiles the generated obfuscated C file with `/O2` and `/MD`. Should any of the steps fail to produce its expected output. The script logs those errors to `compile_tigress.log`. If all obfuscated executables were created successfully, the log is empty.

Transformation	Option	Value
RndArgs	<code>--Exclude</code>	<code>main</code>
RndArgs	<code>--RndArgsBogusNo</code>	<code>2</code>
EncodeLiterals	<code>--EncodeLiteralsKinds</code>	<code>integer</code>
EncodeLiterals	<code>--EncodeLiteralsIntegerKinds</code>	<code>split</code>

Table 4.5: Non default Tigress transformation options used.

4.7 Data Collection Pipeline

The binaries were disassembled by running IDA Pro in headless mode. Once the analysis had completed, it executed a Python script that used IDA’s internal Python API to enumerate the disassembled functions. Only `main` and unrecognized functions had 3-grams extracted from them. This helps reduce confounding with external code because IDA has signatures to recognize common functionality related to compiler inserted subroutines, CRT, etc. IDA is not capable of recognizing custom functions when a binary is compiled without symbols, as is the case here. IDA therefore gives these unrecognized functions a generic name (`sub_xxxxxx`). The full list of blacklisted functions can be seen in Appendix C.

The 3-grams were extracted with the help of a sliding window, because it is a convenient way to extract all sub-sequences of size 3. This aligns with the definition of n -grams [20]. The extraction was performed function by function because it is consistent with other work [14, 15, 19, 57]. The operands were discarded, and only the instruction mnemonics were kept. The 3-grams that were collected from the binary were saved in a CSV file, where the 3-grams were stored in one column and their count in another.

4.8 Metric Usage

One popular method to represent n -gram features is with a vector that contains the number of occurrences [21, 31, 55]. In effect, it can be thought of as a histogram where the n -gram is the bin on the x axis and the y value is the number of times that specific n -gram has been seen. However, such a vector of counts may not be the final representation that is given to the model but instead serves as an initial extraction step. The steps that follow may normalize or weight the counts [55, 69]. In these cases, the final output of such steps is still impacted by the original vector of counts because it is derived from it.

The experiment converts the opcode 3-gram count vector into a probability distribution before drift is measured. Let

$$V = \{c_1, c_2, \dots, c_n\} \quad p_i = \frac{c_i}{\sum_{x=1}^{|V|} c_x}$$

where c_i is the count and p_i is the resulting probability of the i -th opcode 3-gram. This makes it possible to compare the 3-gram composition of different binaries without it being influenced by the absolute instruction count.

For each program, this type of probability distribution is calculated for its baseline binary and its transformed variants. JSD is then computed between the transformed binaries and that program's baseline variant. JSD is therefore used as a model-agnostic way to measure compositional 3-gram changes (drift) induced by code transformations.

4.8.1 RQ1

The computed JSD values support RQ1 by quantifying how much opcode 3-gram drift is induced by the tested compiler flag configurations relative to the baseline binary. RQ1 were answered by reporting, for each program, the mean compiler induced drift across flag configurations. In addition, the standard deviation and the 90th percentile drift are reported. The 90th percentile is reported because it provides insight into the higher end of the drift values without using the most extreme drift result.

For program i , let the compiler induced drift caused by flag configuration $f \in F_2$ be

$$d_{i,f} = JSD(P_{i,baseline}, P_{i,f}), \quad D_i = \{d_{i,f} : f \in F_2\}.$$

then let the mean compiler induced drift and the standard deviation for program i be

$$\bar{d}_i = \frac{1}{|F_2|} \sum_{f \in F_2} d_{i,f} \quad sd_i = \sqrt{\frac{1}{|F_2| - 1} \sum_{f \in F_2} (d_{i,f} - \bar{d}_i)^2}$$

4.8.2 RQ2

RQ2 interprets the JSD values as drift caused by the applied set of Tigress transformations T . The set of Tigress transformations is fixed and repeated over multiple randomized runs. RQ2 therefore reports, for each program, the mean Tigress induced drift across randomized runs. This mean helps reduce the dependence on a single randomized run. In addition, the standard deviation across runs is reported in order to quantify how susceptible the recorded drift is to Tigress randomization.

For program i , let the Tigress induced drift under run $r \in R$ be

$$d_{i,r}^T = JSD(P_{i,baseline}, P_{i,r}^T) \quad D_i^T = \{d_{i,r}^T : r \in R\}$$

the mean drift per program is then

$$\bar{d}_i^T = \frac{1}{|R|} \sum_{r \in R} d_{i,r}^T$$

and the corresponding standard deviation is

$$sd_i^T = \sqrt{\frac{1}{|R| - 1} \sum_{r \in R} (d_{i,r}^T - \bar{d}_i^T)^2}$$

Thus, RQ2 is answered with one mean drift value per program, while the standard deviation helps to show to what extent that mean depends on Tigress randomization.

4.8.3 RQ3

RQ3 compares compiler induced drift with Tigress induced drift for each program. The drift difference is answered with the help of the signed mean difference and is simply $\Delta_i = \bar{d}_i^T - \bar{d}_i$. The drift values are measured in JSD which means that $\Delta_i \in [-1, 1]$ and its result is interpreted as

- The Tigress induced drift is greater on average $\iff \Delta_i > 0$.
- The average Tigress induced drift and compiler induced drift are equal $\iff \Delta_i = 0$.
- The compiler induced drift is greater on average $\iff \Delta_i < 0$.

In addition to the difference in mean drift, RQ3 also reports what the probability is that a randomly chosen Tigress induced drift value exceeds a randomly chosen compiler induced drift value. This was quantified using the probability of superiority, which is estimated by pairwise comparisons of compiler induced and Tigress induced drift values for the same program. For each comparison, a score of 1 is assigned if the Tigress induced drift is greater, 0.5 if the two values are equal, and 0 if the Tigress induced drift is smaller. The probability that Tigress induced drift is greater is then the average of all pairwise comparisons [54]. Let the indicator function be

$$I(A) = \begin{cases} 1, & \text{if } A \text{ is true,} \\ 0, & \text{if } A \text{ is false} \end{cases}$$

The probability is then estimated with $Pr(D_i^T, D_i)$ when $Pr(X, Y)$ is defined as

$$Pr(X, Y) = \frac{1}{|X||Y|} \sum_{x \in X} \sum_{y \in Y} \left[I(x > y) + \frac{1}{2} I(x = y) \right]$$

4.8.4 Relation to RQ4

Because each program was measured repeatedly under the 64 flag configurations in Appendix A, observations within the same program are not independent. This means that the data are multi-leveled, where the individual runs are the lower level observations which are grouped per program ID. In which case a mixed-effect model may be the most appropriate [41].

Repeated-measures (RM) ANVOA is less appropriate here because it assumes sphericity [8, 32]. In the context of this experiment, sphericity means that the variance across programs of the within program drift difference must be the same for every pair of the flag configurations. Sphericity cannot be guaranteed because the risk of violating it increases with the number of repeated measures [32], and if the assumption fails, then the inference can be misleading [8, 41], unless correct adjustments are taken [41].

Least squares linear regression is also not appropriate for this research question. This would force all runs to be treated as independent and ignore the fact that some runs are grouped per program. Doing so can result in incorrect p values. This happens because the standard errors that are computed become incorrect if all fixed

effects are treated as independent variables in a repeated-measures regression setting when they are not [41].

Therefore, RQ4 is answered by fitting a beta mixed effects model. The drift values are the response variables. The eight compiler flags in F_1 are used as fixed effects as they are the variables expected to affect the response variable. Specifically, +1 when a flag is ON and -1 when its OFF. The program id is the grouping factor that serves as the random effect, where $id \in \{1, 2, \dots, 21\}$. The fitted model provides the answer to what the expected change in mean drift is when an individual flag is enabled.

4.9 Validity, Reliability, and Limitations

The reliability of the experiment was improved with the help of automation. Compilation, Tigress transformation, disassembly, 3-gram extraction, and metric computation were all achieved with the help of scripts instead of being done manually. This helps reduce the risk of human induced errors or inconsistent actions between runs. All source code and scripts are publicly available on GitHub [1].

4.9.1 Limitations

The choice to use a fractional factorial design makes it possible to evaluate more flags compared to a full factorial design for the same combinatorial cost. The trade off is that such a design introduces aliasing for three way (and higher) interactions. This trade off is acceptable because none of the research questions directly target those higher order interactions. However, if three way or higher order interactions affect opcode 3-gram drift in any meaningful way, then their influence may be confounded with lower order estimates.

4.9.1.1 Single Feature

The experiment measures drift in a single static feature class, opcode 3-gram distributions. Because it does not include other features commonly used in malware analysis, such as CFG- or API features, the results should only be seen as evidence of drift in opcode 3-grams. It should also not be seen as evidence of drift in n -grams of other sizes.

4.9.1.2 Evasion Evidence

It is not possible to quantify how much observed drift would affect classifier performance because the experiment does not train or test malware classifiers. Instead, any possible robustness degradation is inferred indirectly from previous work. The results should therefore be interpreted as evidence of potential impact and not as proof of evasion against a specific system.

4.9.2 Internal Validity

Multiple actions were taken to strengthen internal validity. Each program had its own baseline variant, created by compiling that program using `/O2`. Then each program variant under test was compared only with that baseline. This reduces the risk that any recorded drift was caused by natural program differences, instead of the applied compiler configuration or Tigress transformation. The study also used a fixed set of tools and architecture. The same version of the tools was used for all runs, and the experiment targets only Windows x86-64. This helps prevent measured drift from being confounded by differences caused by tool versions, different compilers, or different architectures and instructions sets. The full list of software versions can be seen in Appendix D.

4.9.2.1 Compilation

The compilation of the programs was also controlled through an automatic pipeline. Before triggering the CMake configuration, the previous build directory was deleted so that any stored cache variables from previous runs were not part of the current run. Compiler and linker flags were appended to the environment variables in such a way that they overwrite any previous conflicting flags. This guarantees that those flags were, in fact, the options provided to MSVC.

Build failures were captured in logs stored in the same directory as the script file. The build scripts for the baseline programs, the flag configuration programs, and the Tigress programs all have their own log file. The three log files were manually checked for any reported errors after their respective pipelines were finished. The lack of reported errors reduces the risk that failed builds mistakenly enter the final set of artifacts.

4.9.2.2 C Runtime

The C runtime can be linked statically or dynamically, `/MT` and `/MD` respectively. All compilation was done using `/MD` to reduce the risk of confounding the C runtime code with the program's code. However, linking dynamically does not completely eliminate all external code, but it reduces it to a large extent. This helps reduce confounding between module code and external third party code.

4.9.2.3 3-gram Extraction

The disassembling of the binaries and the extraction of their 3-grams is automated, reducing the risk of inconsistencies between runs. The script automatically creates a blacklist of the function names known to IDA. These functions are skipped and do not have their 3-grams extracted. This further reduces the risk of confounding the program code with external code.

The script is tested by a test harness. The test is run on a tiny binary that has had its 3-grams extracted by hand. First, it verifies that the script has successfully created the IDA database file, `.i64`. It then parses the log file produced by IDA for errors. Finally, it compares the script generated 3-grams with that of the expected output.

4.9.2.4 Tigress Functionality Test

Tigress can be brittle because it is a source code generator and is built first and foremost for GCC. Therefore, to ensure that the added obfuscation is functionality preserving, a validation script was run on the obfuscated binaries produced by the Tigress generated source code. The script tests the functionality of each binary by looking for the expected artifacts generated by them.

4.9.2.5 Disassembly

Linear disassembly can fail to properly handle inline data and instead treat it as code. If these data bytes coincide with valid opcodes, the disassembler will output fake instructions. This is unfortunately more common on packed Instruction Set Architectures (ISA) like `x86`, because most byte values are valid opcodes. MSVC sometimes creates inlined data [6], so the use of linear disassemblers, such as `objdump`, is not ideal when working with PE files.

For this reason IDA Pro is used, which is among the most advanced and used disassemblers. IDA Pro disassembles binaries recursively by following its control flow. Recursive disassembly is less likely to output fake instructions, but it is not foolproof. It may struggle with indirect jumps or calls [6]. It can also be fooled using anti-analysis obfuscation [59]. Tigress supports such obfuscation, but it is not applied to any of the transformed binaries as to not break the disassembly process. The use of a recursive disassembler and the lack of anti-analysis obfuscation therefore reduces the risk that the disassembly output is incorrect.

4.9.3 External Validity

The experiment relies on self produced source code modeled after a survey focused on APT attacks. Therefore, the MITRE techniques implemented may be biased towards APT malware and may not fully represent all malware behaviours seen in the wild. Furthermore, the number of programs and their code variety are necessarily limited due to the strict time frame. The implication of this is that behaviour commonly seen in real world malware and legitimate software such as multi-language code bases, custom packers, etc, are not part of the experiment. This means that the programs produced by the modules are more inline with a "toy program", which is a concern raised by De Sutter *et al.* [12]. This means that generalization is limited and that the results may not fully show how the same flags or obfuscations will behave on large malware families seen in the wild.

4.9.3.1 Protection Strength Across Configurations

The method used only tests a small subset of all possible configuration combinations that exist. In reality, Tigress transformations and compiler flags enable a large set of possible configurations that can be manipulated with choice of transformations, choice of flags, options, random seed, etc. This limits generalization to the specific configurations under test.

4.9.3.2 Layered Transformations

The method used applies obfuscation at the source code level, i.e., the compilation stage. It does not apply any obfuscation at the binary level with e.g. packers or crypters. Nor does it apply obfuscation at the assembler stage like what was done by Gibert et al. [21]. This limits the interpretation of the results to binaries that only rely on source code obfuscation and may not properly reflect drift in binaries that use any other layered combination of obfuscation.

4.9.3.3 Tooling

The interpretation of the results are limited to the exact tools used and not to any potential alternatives. The programs are compiled solely with MSVC, therefore any computed drift may not translate to binaries compiled with alternative compilers such as GCC or Clang. This is also true for the obfuscations produced by Tigress, as commercial alternatives exist but are not used, such as Irdeto and Arxan.

The results are first presented as two overall distributions of all observed JSD values, one for flag configurations and one for Tigress runs. The rest of the results are broken down by research question using per-program summaries, heatmaps, and estimates obtained from the beta mixed effects model.

The total distribution of JSD originating from the flag configurations in Figure 5.1 varies from barley measurement to substantial. A noticeable subset of JSD values sits in between the two extremes, in the $[0.20, 0.35]$ range. In contrast, Tigress produced JSD values are much less dispersed and fall within the $[0.42, 0.53]$ range, see Figure 5.2. This indicates that the tested set of transformations, T , is capable of producing high JSD values more reliably compared to the flag configurations tested in F_2 .

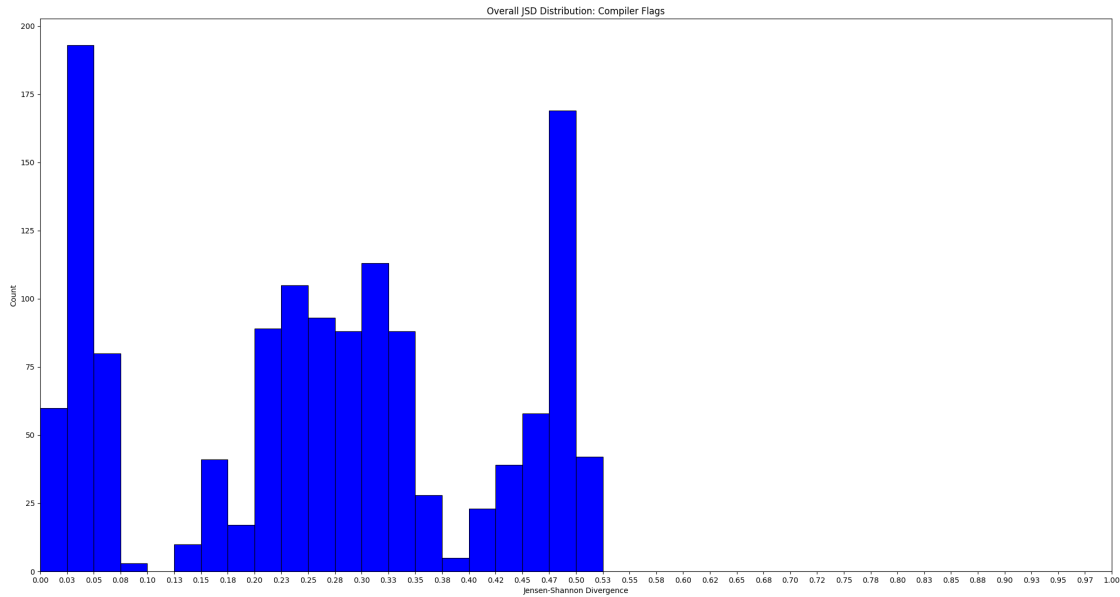


Figure 5.1: Overall compiler induced drift distribution containing all produced JSD values across configurations and programs.

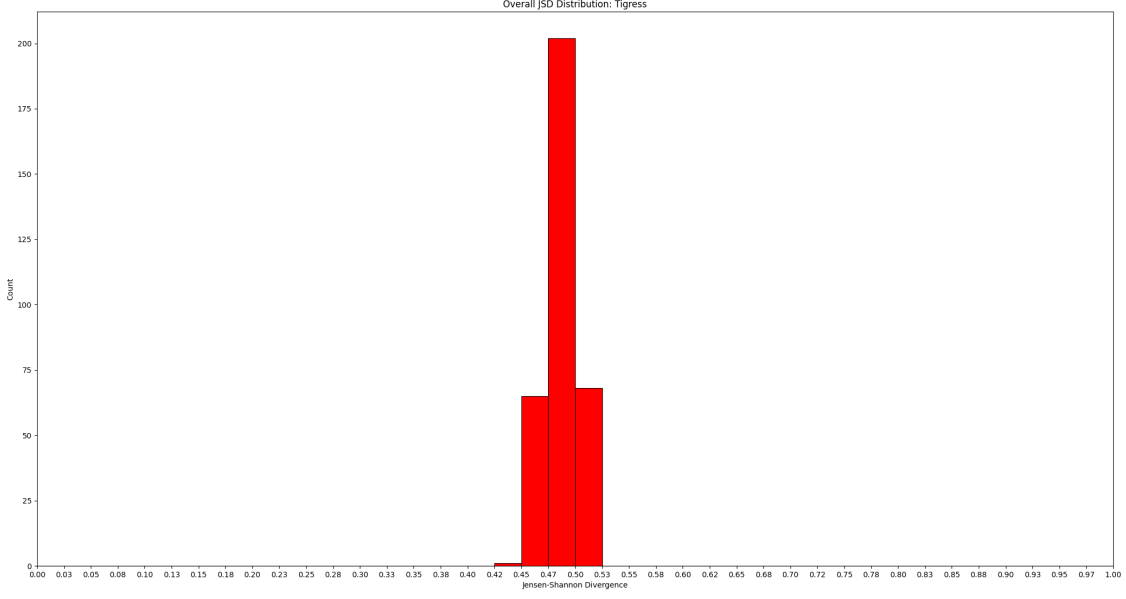


Figure 5.2: Overall Tigress induced drift distribution containing all produced JSD values across randomized runs and programs.

5.1 RQ1

Table 5.1 summarizes the compiler induced opcode 3-gram drift for each of the programs tested. The table reports the mean JSD, the standard deviation, and the 90th percentile across the tested flag configurations. The mean JSD across programs ranges from $[0.239, 0.28]$. However, across programs the 90th percentile ranges from 0.457 to 0.499. This indicates that even though the mean JSD lies in the lower ranges by program, high JSD values exist consistently.

The heatmaps in Figures 5.3 and 5.4 make it clear that the magnitude of the JSD is mainly driven by the flag configuration used and not by program under test. For example, program 1’s JSD values range from 0.08 to 0.501. This suggests that the compilation choices tested are a major factor in the recorded JSD values.

5.1.1 Analysis

The results suggest that compiler induced JSD is not limited to a few outliers. Instead, the flag configurations produce a wide range of JSD values, which includes both low and high outcomes for the same program. This means that opcode 3-gram representations are sensitive to compilation choices when a program’s functionality remains the same and that some of those choices produce substantial opcode 3-gram drift. The answer to RQ1 is therefore that the tested compiler flags produce anywhere from trivial to non-trivial JSD, whose magnitude depends primarily on the chosen flag configuration and secondary on which of the 21 programs is compiled.

Program ID	90th percentile	Mean	SD
1	0.48424138	0.25060220	0.16027373
2	0.47546293	0.24614757	0.15642639
3	0.48468433	0.25317262	0.15990833
4	0.49004197	0.26847400	0.16127628
5	0.46970991	0.24659638	0.15702582
6	0.47462976	0.24933124	0.15312620
7	0.45468261	0.23884583	0.15131356
8	0.49176487	0.27236396	0.16132436
9	0.48924206	0.27090633	0.15874883
10	0.49812699	0.26386584	0.16494068
11	0.49893871	0.27265033	0.16692015
12	0.48114840	0.25366478	0.15806859
13	0.49686683	0.27340108	0.16525503
14	0.49191752	0.26124343	0.16173599
15	0.49756972	0.27345756	0.16517117
16	0.49065087	0.26231985	0.15972890
17	0.47505462	0.25465311	0.15871035
18	0.49255484	0.27522489	0.16270387
19	0.49201658	0.26308962	0.15941190
20	0.49654280	0.27974554	0.16320963
21	0.49879352	0.28084406	0.16317319

Table 5.1: Mean JSD values produced per program across configurations as well as the JSD produced by the 90th percentile flag configurations. Presented numbers are rounded to 8 decimals.

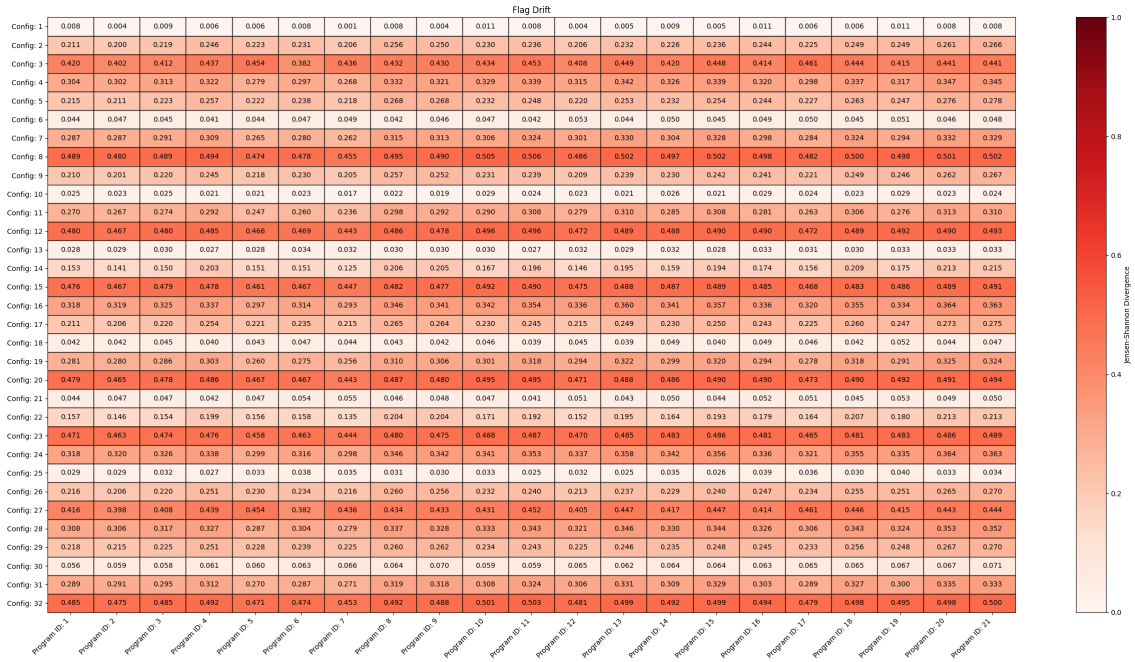


Figure 5.3: Heatmap (1/2) of compiler induced drift for each produced binary. Red is more drift and white is less.

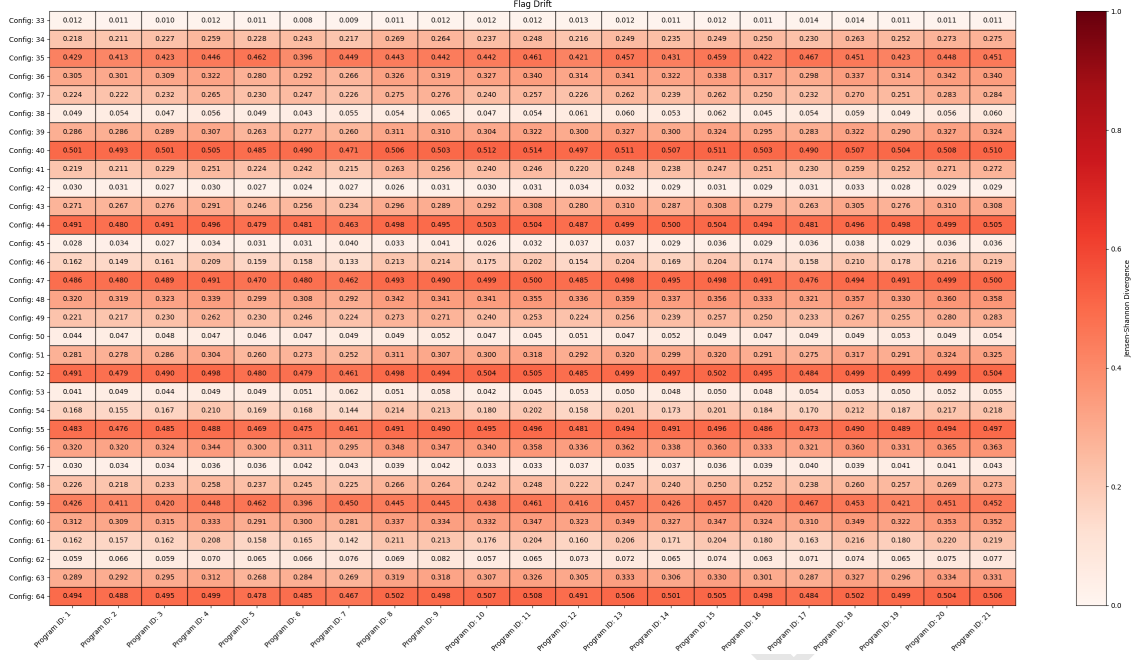


Figure 5.4: Heatmap (2/2) of compiler induced drift for each produced binary. Red is more drift and white is less.

5.2 RQ2

Table 5.2 reports the opcode 3-gram drift induced by the tested Tigress transformation set for each program. For every program, the table reports the mean JSD across the 16 randomized runs, the standard deviation, and the 90th percentile. Across the 21 programs, the mean JSD ranges from 0.460 to 0.508. The standard deviation is small for all programs and falls within the $[0.003, 0.007]$ range. The heatmap in Figure 5.5 shows that the Tigress induced drift is consistently high across programs and runs, and that its produced JSD values have a much smaller spread than the values produced by the flag configurations tested.

5.2.1 Analysis

Taken together, the results show that the tested Tigress transformation set produces substantial opcode 3-gram drift in all the programs used in the experiment. The Tigress induced drift is high and concentrated in a narrow range, unlike its compiler induced counterpart. This indicates that the applied Tigress transformations consistently produce large and stable drift across repeated randomized runs. The answer to RQ2 is therefore that the tested Tigress transformation set causes consistently high opcode 3-gram drift, with mean JSD values never falling below 0.460.

Program ID	90th percentile	Mean	SD
1	0.48111071	0.47452906	0.00593121
2	0.48318983	0.47666487	0.00430497
3	0.48126161	0.47572571	0.00520134
4	0.49944333	0.49566774	0.00299569
5	0.49511395	0.48826981	0.00553671
6	0.49656740	0.49240376	0.00377315
7	0.51263545	0.50795769	0.00461853
8	0.50150802	0.49784099	0.00391484
9	0.51160821	0.50820968	0.00443480
10	0.47663247	0.46682608	0.00734935
11	0.48515304	0.47969799	0.00473402
12	0.47604636	0.47003026	0.00498602
13	0.49517328	0.48816516	0.00433511
14	0.46360920	0.45959607	0.00409740
15	0.49050413	0.48349080	0.00495225
16	0.48969021	0.48420163	0.00492944
17	0.49610118	0.49181285	0.00434270
18	0.50985201	0.50672880	0.00293465
19	0.48633033	0.48170680	0.00409762
20	0.50686338	0.50163471	0.00468185
21	0.49912745	0.49611913	0.00324705

Table 5.2: Mean JSD values produced per program across randomized runs as well as the JSD produced by the 90th percentile randomized runs. Presented numbers are rounded to 8 decimals.

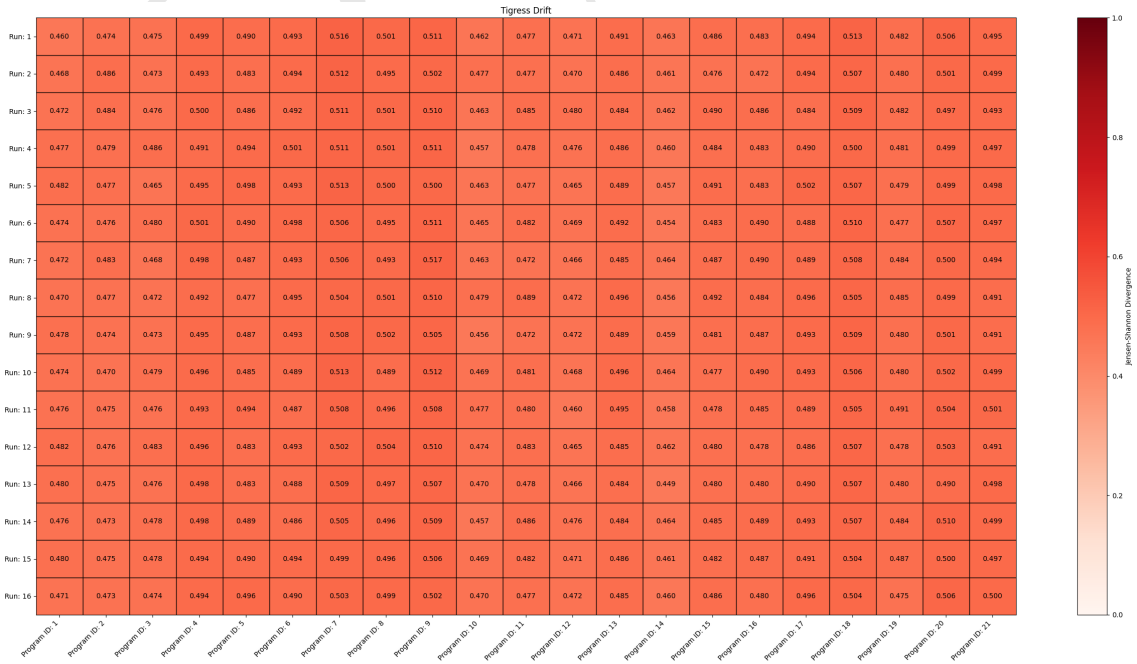


Figure 5.5: Heatmap of Tigress induced drift for each produced binary. Red is more drift and white is less.

5.3 RQ3

Table 5.3 compares compiler induced and Tigress induced opcode 3-gram drift for each program using the signed mean difference and probability superiority. The signed mean difference is positive for all 21 programs, and falls within the range $[0.198, 0.270]$. The probability superiority is above 0.5 for every program, and falls within the range $[0.813, 1.00]$. This means that for every program, the mean Tigress-induced drift is greater than the mean compiler-induced drift. Furthermore, it also implies that a randomly selected Tigress-induced drift value is more likely to be greater than a randomly selected compiler-induced drift value from the same program.

Individually, the highest compiler-induced drift values can compete with the strongest Tigress drift values, as can be seen in Table 5.5. The three highest recorded values were 0.517, 0.516, and 0.514, respectively. The first two are produced by Tigress and the third is produced by a flag configuration. A comparison of the upper-tail likewise indicates that the right compiler flags can rival the drift magnitude caused by Tigress obfuscation, which can be seen in Table 5.4. The table shows that the $|\Delta|$ between the compilers and Tigress 90th percentile is less than 0.1 JSD.

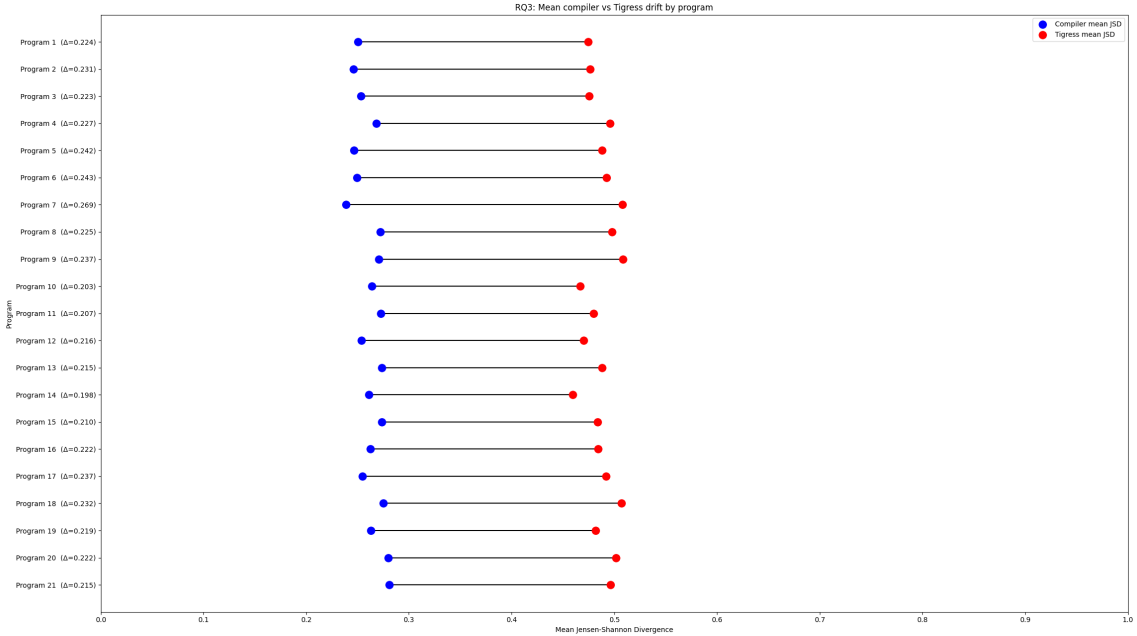


Figure 5.6: Dumbbell plot of mean difference. Red is tigress, blue is compiler flags.

5.3.1 Analysis

Taken together, these results show that a subset of the tested compiler flag configurations do produce opcode 3-gram drift that is comparable to the tested Tigress transformation set. However, the overall results favor Tigress. The signed mean difference is positive for every program, and the probability of superiority values consistently indicates that a randomly chosen Tigress JSD value is more likely to be greater than not compared to a randomly chosen flag configuration JSD value. The

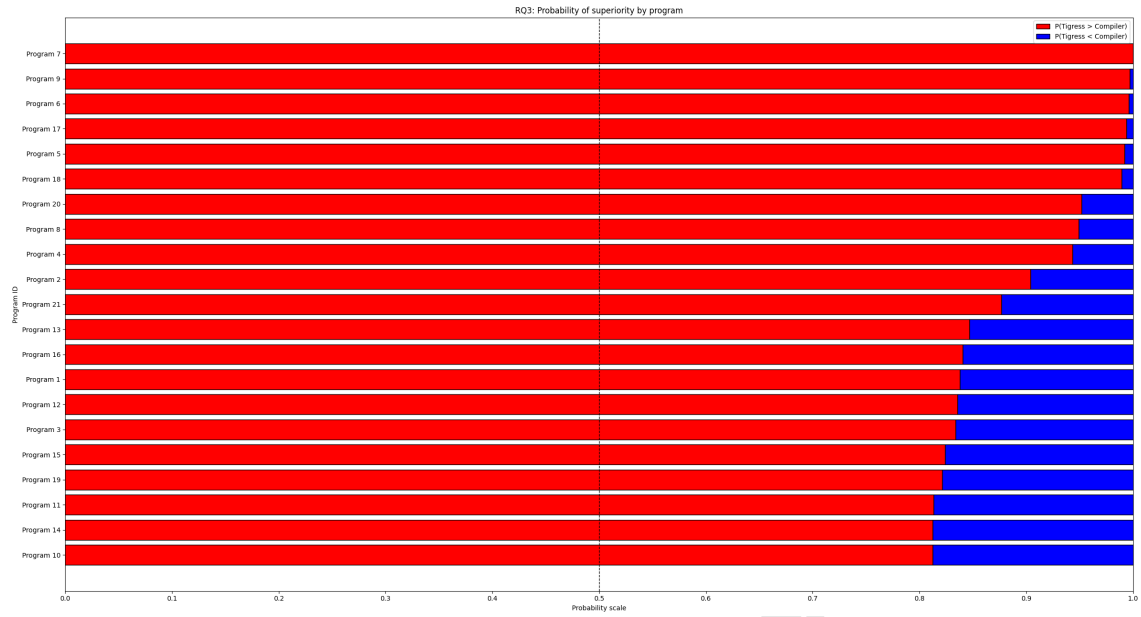


Figure 5.7: Probability of superiority. Red is tigress, blue is compiler flags.

Program ID	Signed Mean Difference	POS
1	0.22392687	0.83789062
2	0.23051730	0.90429688
3	0.22255309	0.83398438
4	0.22719374	0.94335938
5	0.24167343	0.99218750
6	0.24307252	0.99609375
7	0.26911186	1.00000000
8	0.22547703	0.94921875
9	0.23730335	0.99707031
10	0.20296024	0.81250000
11	0.20704766	0.81347656
12	0.21636548	0.83593750
13	0.21476408	0.84667969
14	0.19835265	0.81250000
15	0.21003324	0.82421875
16	0.22188179	0.84082031
17	0.23715974	0.99414062
18	0.23150392	0.98925781
19	0.21861717	0.82128906
20	0.22188917	0.95214844
21	0.21527507	0.87695312

Table 5.3: R3. Presented numbers are rounded to 8 decimals.

Program ID	90th Percentile (T)	90th percentile (C)	Δ
1	0.48111071	0.48424138	-0.00313068
2	0.48318983	0.47546293	0.00772690
3	0.48126161	0.48468433	-0.00342273
4	0.49944333	0.49004197	0.00940135
5	0.49511395	0.46970991	0.02540403
6	0.49656740	0.47462976	0.02193764
7	0.51263545	0.45468261	0.05795284
8	0.50150802	0.49176487	0.00974315
9	0.51160821	0.48924206	0.02236616
10	0.47663247	0.49812699	-0.02149451
11	0.48515304	0.49893871	-0.01378567
12	0.47604636	0.48114840	-0.00510205
13	0.49517328	0.49686683	-0.00169355
14	0.46360920	0.49191752	-0.02830833
15	0.49050413	0.49756972	-0.00706559
16	0.48969021	0.49065087	-0.00096066
17	0.49610118	0.47505462	0.02104656
18	0.50985201	0.49255484	0.01729717
19	0.48633033	0.49201658	-0.00568625
20	0.50686338	0.49654280	0.01032058
21	0.49912745	0.49879352	0.00033393

Table 5.4: 90th percentile drift amount for Tigress (T) and flag configurations (C). Δ represents the signed difference between the two percentiles. Presented numbers are rounded to 8 decimals.

Origin	JSD	Program ID	Configuration/Run
Tigress	0.51734228	9	7
Tigress	0.51610417	7	1
Compiler	0.51414199	11	40

Table 5.5: RQ3 top 3. Presented numbers are rounded to 8 decimals.

answer to RQ3 is therefore that the tested compiler configurations can sometimes produce drift that reaches Tigress' magnitude, but Tigress creates greater drift on average and can do so more consistently across programs and randomized runs.

5.4 RQ4

A beta mixed-effects model with a logit link function was used to answer RQ4. The model was fitted with JSD values as its response variable, the eight compiler flags in F_1 as fixed effects. The enabled flag were modeled as +1 and the disabled flags as -1. The model used the program ID from Table 4.3 as its random effect.

5.4.1 Model Convergence

Effective sample size (ESS) and $\hat{R}$ serve as the initial check of how reliable the model's inference is. The estimates of $\hat{R}$ are trustworthy when the size of ESS is greater than 400 [64], which was true for all fixed effects, see Table 5.6. The fixed effects $\hat{R}$ values in Table 5.6 indicate that the model's eight Markov Chains mixed well. All values are less than 1.01, which is the goal for useful samples [64].

The Monte Carlo standard error (MCSE) represents uncertainty in the Monte Carlo estimate, smaller values mean fewer Monte Carlo errors [64]. The mean MCSE for the models fixed effects are all close to 0, indicating a low amount of Monte Carlo errors.

Parameter	MCSE Mean	ESS Bulk	ESS Tail	$\hat{R}$
/homeparams	0.00002	150338	59584	1.00031
/Qspectre-load	0.00003	143628	60419	1.00011
/Gs0	0.00002	136288	59401	0.99997
/favor:INTEL64	0.00002	167309	56252	1.00009
/Gh	0.00002	149640	58209	1.00012
/QIntel-jcc-erratum	0.00002	167381	58248	1.00009
/guard:cf	0.00002	140942	59602	1.00007
/GL	0.00002	156228	57088	1.00015
Intercept	0.00008	45938	54771	1.00017
1 program_id_sigma	0.00009	31669	43977	1.00008

Table 5.6: Convergence diagnostics data for the beta mixed-effects model fixed effects.

5.4.2 Model Prediction Estimates

Table 5.7 shows the model coefficients on the logit scale [7] and they are not raw JSD values. The mean represents the model's best average estimate of a flag effect, and a smaller standard deviation means that the estimate is more certain. The 95% High Density Interval (HDI) represents a flag coefficients central credible interval. If the interval is evidence of a positive or negative direction of the effect as long as the interval does not cross 0.

Parameter	Mean	SD	HDI 2.5%	HDI 97.5%
/homeparams	0.05531	0.00911	0.03692	0.07263
/Qspectre-load	0.78934	0.00963	0.77060	0.80834
/Gs0	0.03837	0.00911	0.02040	0.05613
/favor:INTEL64	0.00035	0.00908	-0.01731	0.01840
/Gh	0.03261	0.00909	0.01495	0.05045
/QIntel-jcc-erratum	0.01579	0.00914	-0.00204	0.03374
/guard:cf	-0.00305	0.00916	-0.02099	0.01496
/GL	0.52232	0.00919	0.50424	0.54035
Intercept	-1.24641	0.01633	-1.27877	-1.21418
1 program_id_sigma	0.05806	0.01543	0.02954	0.08964

Table 5.7: Fixed-effects coefficient estimates for the beta mixed-effects model. Values are on the link (logit) scale.

5.4.2.1 Strong Positive Effect

Two flags are prominent in Table 5.7 because of their large coefficients, namely /Qspectre-load and /GL. The former has a mean estimated effect of 0.78934 with high certainty because of its low standard deviation. The effect of the flag is correlated with a positive change in JSD with its 95% credible interval leaning strongly in the positive, [0.77060, 0.80834]. The latter, /GL, also has a strong impact on JSD with a mean estimated effect of 0.52232. Its standard deviation is small, which means that the estimate is certain. The credible interval of the latter sits in a slightly lower range [0.50424, 0.54035]. However, it is still confidently positive, which indicates that /GL is also correlated with a positive change in JSD.

5.4.2.2 Small Positive Effect

The credible intervals of the flags /homeparams, /Gs0, and /Gh are all positive, indicating that the estimated effect of these flags increases JSD. However, the lower end of the interval drops very close to 0. For example, the interval of /Gh is [0.01495, 0.05045]. The standard deviation is low for the three flags /homeparams, /Gs0, and /Gh, which means that their estimated effect has high certainty. However, the estimated effect itself is also low 0.05531, 0.03837, and 0.03261, respectively.

5.4.2.3 No Clear Effect

The impact of the remaining three flags, /favor:INTEL64, /QIntel-jcc-erratum, and /guard:cf, is unclear. Their credible intervals range from negative to positive, which means that they cross 0 and that the model does not provide clear evidence that the flags increase or decrease the final JSD.

5.4.3 Analysis

Table 5.8 shows the average marginal predictions for JSD for each $f \in F_1$ when f is toggled ON and OFF. The ON and OFF columns in Table 5.8 provide the average

predicted JSD across the full dataset under these conditions. The column Δ measures the expected change in JSD caused by enabling f .

Flag	ON	OFF	Δ
/Qspectre-load	0.394801	0.126417	0.268384
/GL	0.348517	0.172699	0.175817
/homeparams	0.269771	0.251447	0.018324
/Gs0	0.266964	0.254254	0.012710
/Gh	0.266009	0.255208	0.010801
/QIntel-jcc-erratum	0.263223	0.257993	0.005230
/favor:INTEL64	0.260667	0.260551	0.000116
/guard:cf	0.260103	0.261113	-0.001009

Table 5.8: Average marginal predicted JSD for each flag set to ON or OFF. $\Delta = ON - OFF$.

The increase in the predicted mean JSD is substantially larger for the two flags with strong positive effect, **/Qspectre-load** and **/GL**. The former is predicted to increase JSD by 0.268 on average, while the latter is predicted to increase JSD by 0.176 on average.

/homeparams, **/Gs0** and **/Gh** have a tiny positive average effect on JSD, 0.018, 0.013, and 0.011, respectively. As indicated by their coefficients in Table 5.7. The remaining three flags credible interval crossed the 0 boundary and thus lacked evidence of positive or negative direction. Thus, it is not possible to say whether **/guard:cf** actually causes regression as its Δ suggests.

The answer to RQ4 is that **/Qspectre-load** is the flag that has the largest predicted mean effect on the JSD. Its Δ of 0.268 is approximately 50% greater than that of the runner up, **/GL**.

5.5 Summary

The results showed that both flag configurations and source-level obfuscation can cause significant drift in opcode 3-grams while preserving the program’s functionality. The drift produced by the tested flag configurations ranged from trivial to substantial, and the magnitude of that drift was primarily driven by the chosen configuration. The tested Tigress transformation set produced consistently high drift across programs and randomized runs. Tigress produced a higher mean drift for all 21 programs, compared to the compiler configurations. However, some individual compiler configurations produced drift values close to the strongest Tigress results. The mixed-effects analysis showed that **/Qspectre-load** and **/GL** were the two individual compiler flags that were associated with the strongest increased mean drift, while the remaining flags had only a small or no clear effect.

DRAFT

6.1 Answers to Research Questions

The purpose of the conducted experiment was to examine whether compilation choices could induce meaningful changes in the feature space of opcode 3-grams, while the functionality of the program remained unchanged. Also, if that change was large enough to matter when compared to changes from established source code obfuscation techniques. The core problem was whether compiler configurations could change a commonly used static feature representation enough to impact how compiled binaries should be interpreted in malware analysis research. The findings showed that the extracted opcode 3-gram is not shaped only by program logic, but also by how a program was built.

6.1.1 RQ1

The answer to RQ1 is that the compiler flag configurations tested are capable of inducing opcode 3-gram drift that ranges from trivial to substantial. Plainly stated, recompiling the same program with different compiler flags was enough to modify its opcode 3-gram representation by very different magnitudes. The change in 3-gram representation occurs despite the fact that the functionality of the program remains the same. This means that the compiler was a source of real feature drift in the test environment.

The results highlight an important pattern. That compiler-induced drift was not uniformly strong. The tested compiler options did not push the feature distribution in the same direction or by the same amount. Instead, the effect was broadly dispersed across the tested configurations. Some of the configurations created only small changes in feature space, while others produced much larger changes. This indicates that the strength of compiler-induced drift depends on the chosen flag configuration rather than being universal. The compiler is therefore on its own not a single consistent source of drift, but its impact depends on the specific build choices.

This broad dispersion of drift within the same program is important because it means that variations in opcode 3-grams cannot be explained by differences between the tested programs alone. The general conclusion is that opcode 3-gram representations are vulnerable to build configurations and should therefore not be assumed to be stable representations of program functionality when those configurations are treated as uncontrolled variables.

6.1.2 RQ2

The answer to RQ2 is that the tested Tigress transformation set consistently caused high opcode 3-gram drift. Unlike the compiler, the drift produced by Tigress was not dispersed to the same degree. Instead, it remained concentrated in a high and narrow range across all of the tested programs. That is important because it means that the drift produced by Tigress was disruptive across repeated randomized runs. This shows that the drift caused by the transformation set to the opcode 3-gram representation was done in a consistent manner. That suggests that the observed drift was not primarily caused by the occasional outlier, but by a transformation method that reliably induced drift in opcode 3-grams. This means that Tigress was a systematic source that did not rely on a few lucky runs to create substantial drift.

These findings mean that the tested Tigress transformation set produced drift in a more uniform manner than the tested compiler configurations. Tigress-induced drift is thus high as a general outcome, whereas compiler-induced drift depended heavily on which flag configuration was used. Tigress therefore on average produced a stronger drift and, at the same time, proved to be a more reliable source of opcode 3-gram drift. These results align with the idea that Tigress is an established source of obfuscation and therefore can serve as a useful reference point that compiler-induced drift can be meaningfully compared against.

6.1.3 RQ3

The answer to RQ3 is that a subset of the tested flag configurations can produce drift that is of magnitude comparable to the drift caused by the tested Tigress transformation set. However, when the results are viewed program by program the comparison consistently favors Tigress. For each of the 21 programs, the mean Tigress-induced drift was greater than the mean drift produced by the flag configurations. Thus, Tigress was the stronger overall source of drift of the two under the tested conditions. This means that the compiler and Tigress are not equivalent overall as sources of opcode 3-gram drift, regardless whether both can produce substantial drift.

The average advantage that Tigress has should not be interpreted too strongly, because it would be misleading to conclude that Tigress was stronger in all categories. A comparison of the mean is useful for the full set of results, but it does not present an answer of strong individual compiler outcomes. That distinction is important because the purpose of RQ3 was not solely to determine whether compiler flags outperform Tigress in general (or vice versa). The purpose was also to test whether the tested flag configurations can reach drift magnitudes that rival an established obfuscation source. For that narrower question, the answer is yes.

6.1.3.1 Unexpected Findings

An unexpected finding is that the largest compiler-induced drift values came much closer to Tigress than the average comparison suggests. Although Tigress had a higher mean drift for every program, the comparison of the 90th percentile showed that some flag configurations reached the same magnitude and sometimes exceeded it. Plainly stated, the difference on the high end between the compiler and Tigress was much less one sided than the differences of the averages.

This can also be seen in the ranking of the individual highest recorded drift values. The two largest values came from Tigress, but the third largest came from the compiler. That result shows that individual compiler configurations could compete with the strongest Tigress outcomes, even when the compiler cannot compete on average.

Together, this has two implications. Firstly, Tigress remains the most reliable and stronger source of drift when the full set of observations is taken into account. Secondly, the compiler cannot be ignored as a negligible source of drift because it is lower on average. The right flag configurations were capable of creating drifts of magnitudes similar to Tigress under the tested conditions. Thus, the compiler should be part of the discussion about sources of feature drift.

6.1.4 RQ4

The answer to RQ4 is that the compiler flags in F_1 that make up the tested flag configurations were not equally associated with opcode 3-gram drift. `/Qspectre-load` had the strongest mean association with drift, followed by `/GL`, under the experiments test conditions. The flags `/homeparams`, `/Gs0`, and `/Gh` were associated with a small positive effect, while `/favor:INTEL64`, `/QIntel-jcc-erratum`, and `/guard:cf` were not associated with a clear effect. This means that compiler-induced drift does not increase or decrease uniformly by toggling any of the tested flags. Instead, a small subset of those flags was responsible for most of the drift by making more aggressive changes to the produced code.

6.1.4.1 Spectre Mitigation & Whole Program Optimization

One explanation for the strong result of `/Qspectre-load` is that it is designed to be intrusive. The documentation describes it as having a high performance impact that inserts its security mitigation at every possible load instruction. Like most programs, the programs tested make extensive use of memory locations to store data. When such data are later retrieved to be used by the program, it creates many opportunities for `/Qspectre-load` to change the set of instructions generated. This means that new instructions are inserted broadly across the entire code base, thereby repeatedly affecting opcode order and layout. This is exactly the kind of change to which 3-gram representations are sensitive.

The strong result of `/GL` is therefore expected considering what the flag does. `/GL` enables program optimization across translation units, which means that it does not just make local code changes but changes throughout entire code base. An example of such an optimization is inlining functions across translation units. Plainly stated, it means that the body of the callee can be substituted into each caller, changing the opcode order of the caller as a consequence. The programs used in this thesis relied on multiple C files to organize its code, which means that `/GL` had many opportunities to add its optimizations. The common denominator appears to be that compiler flags that can make instruction selection and replacement across the code base may have a greater likelihood of having a large effect on opcode 3-gram distributions.

6.1.4.2 Parameter Homes, Stack Probes & Custom Profiling

The flags `/homeparams`, `/Gs0`, and `/Gh` all had a positive but weak effect. This suggests that these flags change the generated code, but in a more limited and perhaps repetitive way. `/homeparams` have the ability to affect a large portion of the code base because it changes all functions that take parameters. However, its changes impact the beginning of a function and would mostly consist of inserting `MOV` instructions. The reason is that the shadow stack is always allocated and the parameters are always available in the registers. `/homeparams` simply copies them from the registers to the stack.

Another flag that has the ability to make limited changes to the entire code base is `/Gh`. It affects all functions by inserting a `CALL` instruction at the beginning of a function. `/Gh` calls a custom implementation of `_penter` that has to be fully written in assembly because MSVC does not support inline assembly in x86-64. This is a possible source of endless exotic instructions from a creative adversary. However, this is a local change since it simply adds a new function to the program. This function is therefore unlikely to have a large impact on the final 3-gram distribution, unless the adversary relies on an algorithm to automatically generate a very large amount of exotic junk instructions.

`/Gs0` is similar to the two flags just discussed in that it can make small changes at the beginning of all functions. Here, the condition is that the function make use of local variables. Its changes are tiny and consist of adding a stack probe. This translates into the insertion of a `MOV` and a `CALL` instruction. What these three flags appear to have in common is that they all have the ability to insert one- or two instructions in the beginning of most functions in the code base. This could quickly become repetitive and is unlikely to produce many new distinct 3-grams.

6.1.4.3 No Clear Effect

`/favor:INTEL64`, `/QIntel-jcc-erratum`, and `/guard:cfg` are the remaining three flags and showed no clear effect. That does not prove that they never matter, only that the model did not find evidence of directional effect in this dataset. One possible reason is that their activation is more conditional. `/favor:INTEL64` attempts to optimize for a targeted architecture, but its documentation lacks detailed examples of what those optimizations are. Therefore, its changes may be relatively subtle for the programs tested.

`/QIntel-jcc-erratum` inserts padding to fix alignment issues in specific jump situations, so its effect depends on whether the compiled code actually triggers those conditions often enough. Intel has published a White Paper that describes the issue that the flag exists to solve in more detail [28]. With a better in depth understanding of the issue it could be possible to carefully craft C code that triggers the conditions under which the flag activates. `/guard:cf` adds checks around indirect calls, which means that its impact is tied to where and how often those calls occur. In other words, these flags may be too selective, too limited, or too context driven to produce consistent instruction insertion or reordering.

6.1.4.4 Conclusion

Taken together, these findings indicate that not all compiler flags should be treated as equal sources of opcode 3-gram drift. Under the test conditions used by this experiment, the flags that had the greatest impact were those that either inserted instructions at many locations or changed code generations across a wide scope. This does not necessarily justify a general claim that applies to untested flags, untested compilers, or other feature classes. However, it does highlight that flags could be more likely to impact 3-grams when they change code structure or instruction sequences at scale, and less likely to matter when they are highly localized or conditional.

6.2 Opcode 3-gram Stability

The results of the experiment indicate that opcode 3-grams are not stable representations of the functionality of a program under these test conditions. The core reason is that a program can keep its functionality but still be moved substantially in feature space when recompiled with different flag configurations. Such changes in feature space were not isolated outliers, as the upper-tail of the compiler-induced distribution rivaled the magnitude of Tigress. This means that the extracted opcode 3-gram distribution was not determined only by the logic of a program, but it was also influenced by the build configuration.

This is important because it affects how opcode 3-grams should be interpreted as a representation. If the same underlying program can result in substantially different 3-gram distributions based on compilation choices, then the distribution must be interpreted as representing at least two sources. The first being the implemented logic of the program, and the second how that logic was translated into machine code. The amount each source contributes to the final distribution is likely not constant. In some cases, the logic may dominate the representation, while in others the build settings may have a larger influence as they impact instruction selection, ordering, etc.

A plausible implication of this is that a model trained on opcode 3-grams may be at risk of learning patterns associated with compilation settings instead of malicious or benign program logic. However, the experiment conducted did not test that risk directly with an end-to-end detector, so it should not be treated as an empirical conclusion.

6.3 Relation to Related Work

6.3.1 Gibert et al

The results support the broader point made by Gibert *et al.*, namely that opcode n -gram representations are sensitive to functionality preserving transformations. Gibert *et al.* showed that when the underlying code was changed by obfuscation, some classifiers that relied on opcode n -grams could degrade because the extracted n -gram representation changed even though the functionality of the malware was untouched. This thesis supports their interpretation, as the flag configurations tested were able

to produce measurable and sometimes substantial drift in opcode 3-gram distributions while controlling the program functionality. This further strengthens the idea that opcode 3-grams should not be treated as stable representations of program behaviour, since substantial changes can occur in feature space by other sources.

At the same time, this broadens the set of transformation sources that should be taken seriously in robustness evaluations. In the work by Gibert *et al.* the source was explicit code obfuscation. This fits the traditional view of evasive manipulation. In this thesis however, this type of feature instability occurs as a result of normal compilation choices. This does not in itself prove evasion, since no end-to-end classifier experiment was conducted, and it is likely that the real-world effect would be classifier dependent and impacted by individual models' training choices. However, the results do support the general interpretation that the sensitivity of opcode n -grams is not limited to traditional obfuscation and that normal build choices can change 3-gram distributions enough to become a robustness risk.

6.3.2 Ren *et al*

Ren *et al.* approached the issue of non-default compiler optimizations producing noticeable changes in binaries through the lens of binary diffing, and in part, testing detection on real malware. This thesis complements their paper by providing a quantitative perspective of drift in opcode 3-gram distributions. Specifically, the results provide empirical evidence that flag combinations can induce a wide range of drift in opcode 3-grams under controlled conditions, and thereby support the broad interpretation of Ren *et al.* that compiler choices are a realistic source of change in static features.

Ren *et al.* provided evidence that compiler-induced transformations can bypass detection. The contribution of this thesis is not that it demonstrates evasion, but that it contributes with empirical data of how the representation of a common static feature class can change. Thus, it is narrower and more controlled. By keeping the source program fixed and measuring distribution drift with JSD, the experiment isolates compiler-induced drift that occurs in feature space. That in itself does not prove evasion. However, when interpreted together with the results from previous work that shows that feature drift can degrade detectors. Then it supports the conclusion that compiler flags are a plausible source of robustness problems. A cautious implication is that future work may need to consider compilation choices not only as a source of binary diversity, but also as a potential means of compilation evasion.

6.3.3 Research Gap

The research gap is narrowed but not fully closed by the results produced. What the thesis does is provide empirical evidence that compiler configurations can behave as a source of opcode 3-gram drift in Windows PE binaries. This supports a broad motivation to treat build settings as an experimental variable instead of an implementation detail. From that perspective, the thesis contributes directly to the gap by providing evidence that compiler-induced variation is measurable under the tested conditions.

However, the contribution should be interpreted within its limits. The evidence applies to opcode 3-grams and not n -grams in general, to MSVC and not any compiler, to a subset of flags and not all flags, and to JSD as the chosen model-agnostic drift metric. These limitations mean that the gap has been narrowed but not closed. The results show that the gap is real and worthy of further research, which can collect evidence in areas not covered by this experiment.

6.3.3.1 Limited Data

The limited empirical data of compiler-induced opcode n -gram drift was one part of the identified research gap. This thesis directly contributes to this gap by providing such data for opcode 3-grams. That data was collected through a repeated-measures design that recompiles the same program under controlled conditions and thereby demonstrates a concrete method of quantifying how much the 3-gram distribution can change due to compilation choices.

The contribution therefore consists of two parts. First, the reproducible method for quantifying compiler-induced drift in n -gram distributions. Second, it provides empirical data which show that such drift is not just theoretical but measurable and in some cases capable of reaching non-trivial magnitudes in all tested programs.

6.3.3.2 Lack of Comparisons

The identified research gap also lacked a direct comparison between compiler-induced drift and traditional obfuscation-induced drift. This thesis narrows that gap by providing empirical data from one such comparison. Instead of showing that flag configurations can induce drift, the thesis provides an important context of the magnitude of the recorded drift by comparing it to an established transformation source, Tigris. Without that context, it would be more difficult to judge what are trivial and non-trivial JSD magnitudes.

This magnitude context helps make the research gap clearer and shows its importance. The problem is no longer that compiler settings can change static features, but that those changes can rival an established obfuscation source. This means that the findings extend previous work by showing that everyday build choices belong to the same robustness discussion as established obfuscation. That does not mean that the two should be treated as interchangeable or that variation in build setting and obfuscation will have the same impact on every detector. The conclusion that it does support is that compiler induced binary differences warrant attention in robustness evaluation and dataset design. Ignoring it risks overlooking a source of feature variation that can matter.

6.3.3.3 Model Agnostic Measurements

The scarcity of model-agnostic methods of studying feature drift was another part of the identified research gap. This thesis contributes to this gap by showing the value of measuring changes in feature space under controlled conditions. This is an important alternative in domains that lack standardized benchmarks and where creating paired "original and transformed" binaries may be challenging. End-to-end detector methods are important, but on their own they are not always enough to

isolate contributors to feature drift. The controlled nature of the conducted experiment provides a more controlled view of feature instability than would have been possible in a detector focused methodology.

A model-agnostic approach is less dependent on the existence of standardized benchmarks and less constrained by sharing restrictions. The method used in this thesis makes it possible to study whether a feature representation itself changes under controlled transformations. Previous discussions of benchmarking limitations are extended by these findings by showing that empirical data can still be collected even when end-to-end reproducibility is difficult. The thesis therefore contributes not only by providing empirical data, but also with a model-agnostic methodology that demonstrates how to collect such data.

Interpretations should be made carefully, as model-agnostic drift measurements do not replace classifier evaluations. The reason is that such measurements cannot show by themselves how a specific detector would react to the recorded drift. What such measurements can do is help identify whether substantial feature instability exists in the extracted representations. This makes it useful in conjunction with end-to-end studies rather than as an alternative. The thesis therefore narrows the research gap in regards to the absence of model-agnostic evidence by showing that compiler-induced feature instability in opcode 3-grams can be measured in a controlled way.

6.4 Implications

The primary implication of the results of this thesis is that compiler configurations should not be treated as an implementation detail in studies that rely on static opcode 3-gram features. The presented evidence of substantial feature space instability caused by some of the tested flag configurations matters. The reason being that previous work has shown that instability in static features can be associated with reduced robustness in classifiers, particularly those that use n -gram based representations. Feature drift caused by compiler settings is, therefore, a plausible risk to robustness that researchers should evaluate and not ignore.

An additional implication is if the compiler configurations with the highest risk of inducing drift are ignored by robustness evaluations. Then a detector's reported robustness values may be too optimistic due to potential validity issues. This can make that detector appear robust when tested against an undocumented flag configuration that dominated the training data. However, that same detector may remain vulnerable to feature drift caused by alternative configurations. This may be particularly relevant for feature extraction methods that depend on the program's instructions. Plainly stated, the implication is not that compiler-induced drift can be assumed to always cause evasion, but that it should be treated as a question that must be tested.

Taken together, these implications suggest that research should focus more on how binaries are produced when studying the robustness of trained models in static malware analysis. Future robustness evaluations and datasets should therefore be designed with the diversity of compilation taken into account, or risk incorrectly explaining robustness degradation as caused by new functionality or obfuscation.

6.4.1 Robustness Evaluation

It is not enough to evaluate robustness only against dedicated obfuscators, even though that remains important. Evaluations should also look at flag configurations or risk overlooking a source of feature instability. Compiler-induced feature drift should be considered part of the robustness problem itself. Especially if the evaluated detector was trained with opcode 3-grams. Plainly stated, a complete evaluation should test both dedicated obfuscators and normal build diversity, since both can alter the extracted 3-gram representation without changing the functionality.

However, these evaluations do not necessarily need to treat all available compiler flags equally. The results of RQ4, indicate that the flags tested did not contribute an equal amount to the overall drift and that some configurations were more important than others. Consequently, future robustness testing can prioritize a subset of flags and flag combinations that are more influential. Not having to assume that all compiler configurations are equally important makes these additional evaluations more manageable by emphasizing targeted evaluation instead of complete enumeration.

6.4.2 Dataset Construction

There are also implications for how datasets for static malware analysis should be constructed and documented. If the compiler settings can systematically manipulate opcode 3-gram distributions, then the documentation of a dataset should not just contain labels, family membership, etc. Those remain important but are not in themselves sufficient to describe all sources that may impact the shape of a binaries feature space. Compilation settings should therefore be treated as relevant origin information for datasets that contain static binary features.

This is of direct importance for how the quality of a dataset should be calculated. A detector trained on samples with an undocumented set of compiler settings may see its performance degrade when exposed to binaries compiled with a different set of settings. This performance degradation may then be incorrectly attributed to reasons other than compiler-induced drift. The implication of this is that datasets that lack diverse compilation settings may contain a source of dataset bias. Compilation diversity is therefore important not only for robustness evaluations, but also for dataset design.

The construction of future datasets should therefore contain compilation metadata whenever it is known. This does not necessarily mean that every available compiler setting must be captured, but it does mean that data related to build conditions should be reported so that others can understand to what extent compilation diversity is represented. When binaries are generated from source code, such metadata may help make the feature space of the dataset clearer. The datasets constructed from purely existing binary collections should at least recognize that the absence of such metadata is a limitation. Since it becomes more difficult to judge whether the available features in the dataset represent realistic binary variability.

6.4.2.1 Individual Flags

As mentioned in previous sections, the results of RQ4 showed that not all the flags in F_1 were associated with the same mean drift. As a result, some flags are more

important than others when constructing datasets or performing robustness evaluations. The real-world consequences of this are that if some flags contribute much more strongly to opcode 3-gram drift than others, then future work may not need to represent every available flag. Instead, focus can be given to the flags and flag combinations that have the strongest impact on extracted representations.

In terms of dataset diversification, the solution is likely not to arbitrarily represent a wide variety of compiler settings. What is more important is whether the represented settings contain the types of build variability that systematically make changes in feature space. In that sense, flags with weak or unclear association may not need the same representation as those whose association is strong. As with robustness evaluation, this also highlights a targeted strategy in which the representation of compilation diversity is driven by measured feature drift instead of the number of flag configurations.

6.5 Trustworthiness

A core driver of trust is that the experiment conducted repeated comparisons between a transformed program variant and that program’s baseline. Instead of relying on a single comparison between unrelated programs. This means that any recorded drift is less likely to originate from normal differences between programs and more likely to be explained by the applied transformation. The trustworthiness is further strengthened by the fact that the observed patterns were consistent across the experiment and not the result of a single isolated observation.

The fact that Tigress produced a higher drift on average, while the two 90th percentiles produced almost identical values makes the comparison more convincing. The reason is that it suggests that the conclusion does not depend on a single program that acts as an outlier or some kind of anomalous transformation. This is also supported by the overall drift distributions, compiler-induced drift covers a wide range while Tigress-induced drift is narrow and consistently high.

The thesis predicted in the problem statement that compilation settings could induce change in opcode n -gram distributions and that these changes could matter for the evaluation of robustness and the construction of datasets. The results align with that prediction by providing empirical support for that concern, but also suggest that detector stability may be more impacted by build diversity than is often implicitly assumed. Finally, the results are the most trustworthy when interpreted within their limitations.

6.5.1 Limitations

Any conclusions drawn from the results presented in this thesis should be interpreted within the boundaries of its scope. First, the results are specifically related to opcode 3-grams and may not generalize to any n . It also does not mean that the same amount of drift can be assumed to occur in other static feature classes, such as byte n -grams, API-calls, CFG related features, etc. Plainly stated, the thesis provides evidence of instability in opcode 3-grams under strict testing conditions and not universal truth about all static binary features.

Second, the compiler-induced drift results are limited to the subset of flags tested. This means that the complete drift potential by MSVC remains an open question. In addition, the results relate only to MSVC and should not be assumed to also hold true for other compilers such as GCC or Clang. This is true even if other compilers have flags that are similar to the tested subset. The reason for this is that different compilers can apply different internal transformations and therefore produce different instruction outcomes.

The experiment was conducted on `x86-64`, which is another limitation. The results may not generalize to other architectures such as `ARM`. The reason is because of instruction sets differences, but also because the same compiler may use different code generation or optimization strategies. This means that the results should be understood as drift induced by MSVC's `x86-64` toolset and not as any MSVC toolset.

The controlled source code itself places a limitation on how far the results can be generalized. The interpretation that instability can result from repeatedly recompiling the same source holds for smaller controlled code samples. However, the results do not show how stability will behave for large and diverse codebases under the same testing conditions. A large codebase may allow for additional interactions between optimization, control flow, or code layout that smaller programs do not. The results are therefore evidence that compiler-induced drift exists and can be substantial, and not a ground truth of how large those drift estimates are in every context.

Finally, the consistency and drift magnitude produced by Tigress may vary when its transformation settings change. This could be changes to the number of splits or other command line options, as well as adding or subtracting individual transformations. RQ3 therefore provides context relative to the tested Tigress conditions, and is not a ground truth value for all possible Tigress transformations. Overall, these limitations do not invalidate the thesis conclusion, but instead delimit the boundary of where that conclusion lives. That boundary is clear, the results provides credible evidence of non-trivial opcode 3-gram drift under recompilation with MSVC and what remains is the question of how strongly that type of feature instability extends to other feature classes, compilers, architectures, and transformation settings.

DRAFT

Chapter 7

Conclusions and Future Work

7.1 Future Work

TODO: Perhaps some flags can be normalized away during feature extraction. E.g. LFENCE

TODO: collect data for GCC and Clang.

TODO: collect data for more compiler flags.

TODO: Malware can arguably not be assumed to default to O2 because malware does not care about performance.

TODO: Malware don't care about performance because using abundance of hardware resources creates noise which may alert the victim. This is counterproductive because malware wants to stay hidden.

TODO: One possible exception is ransomware type which has a one time payload which quickly needs to finish before the victim can react.

DRAFT

References

- [1] master_thesis. [Online]. Available: https://github.com/PackedCrack/master_thesis
- [2] (2026) Malware. AV-TEST. [Online]. Available: <https://www.av-test.org/en/statistics/malware/>
- [3] A. Akshintala, B. Jain, C.-C. Tsai, M. Ferdman, and D. E. Porter, “x86-64 instruction usage among c/c++ applications,” in *12th ACM International Conference on Systems and Storage*. New York, NY, USA: ACM, 2019, pp. 68–79.
- [4] A. Al-Dujaili, A. Huang, E. Hemberg, and U.-M. O’Reilly, “Adversarial deep learning for robust detection of binary encoded malware,” in *2018 IEEE Security and Privacy Workshops (SPW)*, 2018, pp. 76–82.
- [5] H. S. Anderson and P. Roth, “Ember: An open dataset for training static pe malware machine learning models,” *arXiv preprint arXiv:1804.04637*, 2018.
- [6] D. Andriesse, *Practical Binary Analysis: Build Your Own Linux Tools for Binary Instrumentation, Analysis, and Disassembly*. No Starch Press, 2018.
- [7] Bambi. Plot slopes. [Online]. Available: https://bambinos.github.io/bambi/notebooks/plot_slopes.html
- [8] M. J. Blanca, J. Arnau, F. J. García-Castro, R. Alarcón, and R. Bono, “Repeated measures anova and adjusted f-tests when sphericity is violated: which procedure is best?” *Frontiers in psychology*, vol. 14, 2023.
- [9] S. Chandran, S. R. Syam, S. Sankaran, T. Pandey, and K. Achuthan, “From static to ai-driven detection: A comprehensive review of obfuscated malware techniques,” *IEEE Access*, vol. 13, 2025.
- [10] L. Chen, Y. Ye, and T. Bourlai, “Adversarial machine learning in malware detection: Arms race between evasion attack and defense,” in *2017 European Intelligence and Security Informatics Conference (EISIC)*, 2017, pp. 99–106.
- [11] H. Darabian, S. Homayounoot, A. Dehghantanha, S. Hashemi, H. Karimipour, R. M. Parizi, and K.-K. R. Choo, “Detecting cryptomining malware: a deep learning approach for static and dynamic analysis,” *Journal of Grid Computing*, vol. 18, pp. 293–303, 2020.
- [12] B. De Sutter, S. Schrittwieser, B. Coppens, and P. Kochberger, “Evaluation methodologies in software protection research,” *ACM Comput. Surv.*, vol. 57, no. 4, Dec. 2024. [Online]. Available: <https://doi.org/10.1145/3702314>

- [13] A. S. Filho, R. J. Rodríguez, and E. L. Feitosa, “Evasion and countermeasures techniques to detect dynamic binary instrumentation frameworks,” *Digital Threats*, vol. 3, no. 2, Feb. 2022. [Online]. Available: <https://doi.org/10.1145/3480463>
- [14] M. Gaber, M. Ahmed, and H. Janicke, “Zero day malware detection with alpha: Fast dbi with transformer models for real world application,” *Computers & electrical engineering*, vol. 128, 2025.
- [15] —, “Zero day ransomware detection with pulse: Function classification with transformer models and assembly language,” *Computers & security*, vol. 148, 2025.
- [16] M. G. Gaber, M. Ahmed, and H. Janicke, “Malware detection with artificial intelligence: A systematic literature review,” *ACM Comput. Surv.*, vol. 56, no. 6, Jan. 2024. [Online]. Available: <https://doi-org.miman.bib.bth.se/10.1145/3638552>
- [17] N. Galloro, M. Polino, M. Carminati, A. Continella, and S. Zanero, “A systematical and longitudinal study of evasive behaviors in windows malware,” *Computers & Security*, vol. 113, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404821003746>
- [18] J. Geng, J. Wang, Z. Fang, Y. Zhou, D. Wu, and W. Ge, “A survey of strategy-driven evasion methods for pe malware: Transformation, concealment, and attack,” *Computers & Security*, vol. 137, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404823005059>
- [19] D. Gibert, C. Mateu, and J. Planes, “A hierarchical convolutional neural network for malware classification,” in *2019 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2019, pp. 1–8.
- [20] —, “The rise of machine learning for detection and classification of malware: Research developments, trends and challenges,” *Journal of Network and Computer Applications*, vol. 153, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1084804519303868>
- [21] D. Gibert, C. Mateu, J. Planes, and J. Marques-Silva, “Auditing static machine learning anti-malware tools against metamorphic attacks,” *Computers & Security*, vol. 102, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404820304326>
- [22] Github. Babukransomwaresourcecode. [Online]. Available: <https://github.com/Hildaboo/BabukRansomwareSourceCode>
- [23] —. Mirai-source-code. [Online]. Available: <https://github.com/jgamblin/Mirai-Source-Code>
- [24] —. Silverrat-full-source-code_1. [Online]. Available: https://github.com/gavz/SilverRAT-FULL-Source-Code_1
- [25] —. Tinynuke. [Online]. Available: <https://github.com/rossja/TinyNuke>
- [26] R. Harang and E. M. Rudd, “Sorel-20m: A large scale benchmark dataset for malicious pe detection,” *arxiv:cs.CR/2012.07634*, 2020.

- [27] IBM. (2022) Cost of a data breach report 2022. [Online]. Available: <https://web.archive.org/web/20230813022133/https://www.ibm.com/downloads/cas/%203R8N1DZJ>
- [28] Intel. (2019) Intel mitigations for jump conditional code erratum. [Online]. Available: <https://www.intel.com/content/www/us/en/content-details/841076/intel-mitigations-for-jump-conditional-code-erratum.html>
- [29] Kaspersky. (2024) The cyber surge: Kaspersky detected 467,000 malicious files daily in 2024. [Online]. Available: <https://www.kaspersky.com/about/press-releases/the-cyber-surge-kaspersky-detected-467000-malicious-files-daily-in-2024>
- [30] S. Kim, S. Yeom, H. Oh, D. Shin, and D. Shin, “Automatic malicious code classification system through static analysis using machine learning,” *Symmetry*, vol. 13, no. 1, 2021. [Online]. Available: <https://www.mdpi.com/2073-8994/13/1/35>
- [31] S. M. Ko, J. Yang, T. Kim, and I. You, “N-gram opcode frequency based malware detection using cnn algorithm: N-gram opcode frequency based malware,” *Soft computing (Berlin, Germany)*, vol. 29, no. 8, pp. 4045–4053, 2025.
- [32] Laerd Statistics. Sphericity. [Online]. Available: <https://statistics.laerd.com/statistical-guides/sphericity-statistical-guide.php>
- [33] D. Li, Q. Li, Y. . Ye, and S. Xu, “Arms race in adversarial malware detection: A survey,” *ACM computing surveys*, vol. 55, no. 1, pp. 1–35, 2023.
- [34] Lukas Meier. Incomplete block designs. [Online]. Available: <https://people.math.ethz.ch/~meier/teaching/anova/incomplete-block-designs.html>
- [35] D. Maiorca, B. Biggio, and G. Giacinto, “Towards adversarial malware detection: Lessons learned from pdf-based attacks,” *ACM Comput. Surv.*, vol. 52, no. 4, Aug. 2019. [Online]. Available: <https://doi.org/10.1145/3332184>
- [36] E. Marin, M. Almukaynizi, E. Nunes, and P. Shakarian, “Community finding of malware and exploit vendors on darkweb marketplaces,” in *2018 1st International Conference on Data Intelligence and Security (ICDIS)*. IEEE, 2018, pp. 81–84.
- [37] L. F. Martín Liras, A. R. de Soto, and M. A. Prada, “Feature analysis for data-driven apt-related malware discrimination,” *Computers & Security*, vol. 104, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404821000262>
- [38] Microsoft. Cl environment variables. [Online]. Available: <https://learn.microsoft.com/en-us/cpp/build/reference/cl-environment-variables?view=msvc-170>
- [39] Microsoft. Compiler options listed by category. [Online]. Available: <https://learn.microsoft.com/en-us/cpp/build/reference/compiler-options-listed-by-category?view=msvc-170>

- [40] —. /favor (optimize for architecture specifics). [Online]. Available: <https://learn.microsoft.com/en-us/cpp/build/reference/favor-optimize-for-architecture-specifics?view=msvc-170#remarks>
- [41] V. F. Misangyi, J. A. LePine, J. Algina, and F. Goeddeke, “The adequacy of repeated-measures regression for multilevel research: Comparisons with repeated-measures anova, multivariate repeated-measures anova, and multilevel modeling across various multilevel research designs,” *Organizational research methods*, vol. 9, no. 1, pp. 5–28, 2006.
- [42] M. Nunes, P. Burnap, P. Reinecke, and K. Lloyd, “Bane or boon: Measuring the effect of evasive malware on system call classifiers,” *Journal of Information Security and Applications*, vol. 67, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2214212622000813>
- [43] U. of Arizona. Encode literals. [Online]. Available: <https://tigress.wtf/encodeLiterals.html>
- [44] —. Flatten. [Online]. Available: <https://tigress.wtf/flatten.html>
- [45] —. Randomize arguments. [Online]. Available: <https://tigress.wtf/randomizeArguments.html>
- [46] —. Split. [Online]. Available: <https://tigress.wtf/split.html>
- [47] N. I. of Standards and Technology. Fractional factorial design specifications and design resolution. [Online]. Available: <https://www.itl.nist.gov/div898/handbook/pri/section3/pri3344.htm>
- [48] J. PARK, Y. . JANG, S. HONG, and Y. PARK, “Automatic detection and bypassing of anti-debugging techniques for microsoft windows environments,” *Advances in electrical and computer engineering*, vol. 19, no. 2, pp. 23–28, 2019.
- [49] E. Raff, R. Zak, R. Cox, J. Sylvester, P. Yacci, R. Ward, A. Tracy, M. McLean, and C. Nicholas, “An investigation of byte n-gram features for malware classification,” *Journal of Computer Virology and Hacking Techniques*, vol. 14, no. 1, pp. 1–20, 2018.
- [50] M. R. Rahman, S. K. Basak, R. Mahdavi Hezaveh, and L. Williams, “Sok: An empirical investigation of malware techniques in advanced persistent threat attacks,” *Computers & Security*, vol. 157, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404825003074>
- [51] X. Ren, M. Ho, J. Ming, Y. Lei, and L. Li, “Unleashing the hidden power of compiler optimization on binary code difference: an empirical study,” in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. New York, NY, USA: Association for Computing Machinery, 2021, p. 142–157. [Online]. Available: <https://doi.org/10.1145/3453483.3454035>
- [52] X. Ren, M. Ren, Y. Lei, and J. Ming, “Revisiting optimization-resilience claims in binary diffing tools: Insights from llvm peephole optimization analysis,” *Proceedings of the ACM on software engineering*, vol. 2, no. FSE, pp. 2689–2711, 2025.

- [53] R. Ronen, M. Radu, C. Feuerstein, E. Yom-Tov, and M. Ahmadi, “Microsoft malware classification challenge,” *ArXiv e-prints*, 2018.
- [54] J. Ruscio and B. L. Gera, “Generalizations and extensions of the probability of superiority effect size estimator,” *Multivariate behavioral research*, vol. 48, no. 2, pp. 208–219, 2013.
- [55] I. Santos, F. Brezo, X. Ugarte-Pedrero, and P. G. Bringas, “Opcode sequences as representation of executables for data-mining-based unknown malware detection,” *Information sciences*, vol. 231, pp. 64–82, 2013.
- [56] S. Schrittwieser, S. Katzenbeisser, J. Kinder, G. Merzdovnik, and E. Weippl, “Protecting software through obfuscation: Can it keep pace with progress in code analysis?” *ACM computing surveys*, vol. 49, no. 1, pp. 1–37, 2016.
- [57] J. Sexton, C. Storlie, and B. Anderson, “Subroutine based detection of apt malware,” *Journal of Computer Virology and Hacking Techniques*, vol. 12, no. 4, pp. 225–233, 2016.
- [58] S. Sharmeen, Y. A. Ahmed, S. Huda, B. S. Kocer, and M. M. Hassan, “Avoiding future digital extortion through robust protection against ransomware threats using deep learning based adaptive approaches,” *IEEE access*, vol. 8, pp. 24 522–24 534, 2020.
- [59] M. Sikorski and A. Honig, *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press, 2012.
- [60] The Pennsylvania State University. 4.7 – incomplete block designs. [Online]. Available: <https://online.stat.psu.edu/stat503/lesson/4/4.7>
- [61] D. Ucci, L. Aniello, and R. Baldoni, “Survey of machine learning techniques for malware analysis,” *Computers & Security*, vol. 81, pp. 123–147, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404818303808>
- [62] —, “Survey of machine learning techniques for malware analysis,” *Computers & security*, vol. 81, pp. 123–147, 2019.
- [63] University of Arizona. Home. [Online]. Available: <https://tigress.wtf/>
- [64] A. Vehtari, A. Gelman, D. Simpson, B. Carpenter, and P.-C. Bürkner, “Rank-normalization, folding, and localization: An improved $\hat{r}$ for assessing convergence of mcmc (with discussion),” *Bayesian analysis*, vol. 16, no. 2, 2021.
- [65] R. Yamagishi, S. Fujii, and T. Sato, “Cultivating skilled malware analysts: Clarification of practical malware dynamic analysis through interviews,” *International journal of information security*, vol. 24, no. 137, 2025.
- [66] L. Yang, A. Ciptadi, I. Laziuk, A. Ahmadzadeh, and G. Wang, “Bodmas: An open dataset for learning based temporal analysis of pe malware,” in *2021 IEEE Security and Privacy Workshops (SPW)*. IEEE, 2021, pp. 78–84.
- [67] Y. Ye, T. Li, D. Adjeroh, and S. S. Iyengar, “A survey on malware detection using data mining techniques,” *ACM Comput. Surv.*, vol. 50, no. 3, Jun. 2017. [Online]. Available: <https://doi-org.miman.bib.bth.se/10.1145/3073559>

- [68] Yibi Huang, University of Chicago. (2022) Stat 222 lecture 20–21: Incomplete block designs. [Online]. Available: https://www.stat.uchicago.edu/~yibi/teaching/stat222/2022/L20_21.pdf
- [69] H. Zhang, X. Xiao, F. Mercaldo, S. Ni, F. Martinelli, and A. K. Sangaiah, “Classification of ransomware families with machine learning based on n -gram of opcodes,” *Future generation computer systems*, vol. 90, pp. 211–221, 2019.

Appendix A

T1005: File Extensions

Documents

.ppt	.pptx	.pptm	.pot
.potx	.odp	.otp	.ods
.doc	.docx	.dot	.dotx
.odt	.ott	.rtf	.txt
.pdf	.tsv	.csv	.ots
.xls	.xlsx	.xlsm	.xlt
.xltx			

Images

.jpg	.jpeg	.jpe	.png
.bmp	.gif	.tif	.tiff
.webp	.heic	.heif	.ico
.svg			

Source code

.c	.h	.cpp	.hpp
.cc	.cxx	.hh	.hxx
.cs	.java	.kt	.swift
.m	.mm	.py	.pyw
.ipynb	.js	.mjs	.cjs
.ts	.tsx	.go	.rs
.lua	.rb	.php	.sh
.bash	.zsh	.ps1	.bat
.cmd	.sql	.html	.htm
.css	.scss	.xml	.json
.yaml	.yml	.toml	.make
.mk	.cmake	.gradle	

Video

.mp4	.m4v	.mkv	.webm
.avi	.mov	.qt	.wmv
.flv	.f4v	.mpeg	.mpg
.m2v	.3gp	.3g2	.ts
.m2ts			

DRAFT

Appendix A

Fractional Factorial Design Table

Run	/homeparams	/Qspectre-load	/Gs0	/favor:INTEL64	/Gh	/QIntel-jcc-erratum	/guard:cf	/GL
1	OFF	OFF	OFF	OFF	OFF	OFF	ON	OFF
2	ON	OFF	OFF	OFF	OFF	OFF	OFF	ON
3	OFF	ON	OFF	OFF	OFF	OFF	OFF	ON
4	ON	ON	OFF	OFF	OFF	OFF	ON	OFF
5	OFF	OFF	ON	OFF	OFF	OFF	OFF	ON
6	ON	OFF	ON	OFF	OFF	OFF	ON	OFF
7	OFF	ON	ON	OFF	OFF	OFF	ON	OFF
8	ON	ON	ON	OFF	OFF	OFF	OFF	ON
9	OFF	OFF	OFF	ON	OFF	OFF	OFF	ON
10	ON	OFF	OFF	ON	OFF	OFF	ON	OFF
11	OFF	ON	OFF	ON	OFF	OFF	ON	OFF
12	ON	ON	OFF	ON	OFF	OFF	OFF	ON
13	OFF	OFF	ON	ON	OFF	OFF	ON	OFF
14	ON	OFF	ON	ON	OFF	OFF	OFF	ON
15	OFF	ON	ON	ON	OFF	OFF	OFF	ON
16	ON	ON	ON	ON	OFF	OFF	ON	OFF
17	OFF	OFF	OFF	OFF	ON	OFF	OFF	ON
18	ON	OFF	OFF	OFF	ON	OFF	ON	OFF
19	OFF	ON	OFF	OFF	ON	OFF	ON	OFF
20	ON	ON	OFF	OFF	ON	OFF	OFF	ON
21	OFF	OFF	ON	OFF	ON	OFF	ON	OFF
22	ON	OFF	ON	OFF	ON	OFF	OFF	ON
23	OFF	ON	ON	OFF	ON	OFF	OFF	ON
24	ON	ON	ON	OFF	ON	OFF	ON	OFF
25	OFF	OFF	OFF	ON	ON	OFF	ON	OFF
26	ON	OFF	OFF	ON	ON	OFF	OFF	ON
27	OFF	ON	OFF	ON	ON	OFF	OFF	ON
28	ON	ON	OFF	ON	ON	OFF	ON	OFF
29	OFF	OFF	ON	ON	ON	OFF	OFF	ON
30	ON	OFF	ON	ON	ON	OFF	ON	OFF

Run	/homeparams	/Qspectre-load	/Gs0	/favor:INTEL64	/Gh	/QIntel-jcc-erratum	/guard:cf	/GL
31	OFF	ON	ON	ON	ON	OFF	ON	OFF
32	ON	ON	ON	ON	ON	OFF	OFF	ON
33	OFF	OFF	OFF	OFF	OFF	ON	OFF	OFF
34	ON	OFF	OFF	OFF	OFF	ON	ON	ON
35	OFF	ON	OFF	OFF	OFF	ON	ON	ON
36	ON	ON	OFF	OFF	OFF	ON	OFF	OFF
37	OFF	OFF	ON	OFF	OFF	ON	ON	ON
38	ON	OFF	ON	OFF	OFF	ON	OFF	OFF
39	OFF	ON	ON	OFF	OFF	ON	OFF	OFF
40	ON	ON	ON	OFF	OFF	ON	ON	ON
41	OFF	OFF	OFF	ON	OFF	ON	ON	ON
42	ON	OFF	OFF	ON	OFF	ON	OFF	OFF
43	OFF	ON	OFF	ON	OFF	ON	OFF	OFF
44	ON	ON	OFF	ON	OFF	ON	ON	ON
45	OFF	OFF	ON	ON	OFF	ON	OFF	OFF
46	ON	OFF	ON	ON	OFF	ON	ON	ON
47	OFF	ON	ON	ON	OFF	ON	ON	ON
48	ON	ON	ON	ON	OFF	ON	OFF	OFF
49	OFF	OFF	OFF	OFF	ON	ON	ON	ON
50	ON	OFF	OFF	OFF	ON	ON	OFF	OFF
51	OFF	ON	OFF	OFF	ON	ON	OFF	OFF
52	ON	ON	OFF	OFF	ON	ON	ON	ON
53	OFF	OFF	ON	OFF	ON	ON	OFF	OFF
54	ON	OFF	ON	OFF	ON	ON	ON	ON
55	OFF	ON	ON	OFF	ON	ON	ON	ON
56	ON	ON	ON	OFF	ON	ON	OFF	OFF
57	OFF	OFF	OFF	ON	ON	ON	OFF	OFF
58	ON	OFF	OFF	ON	ON	ON	ON	ON
59	OFF	ON	OFF	ON	ON	ON	ON	ON
60	ON	ON	OFF	ON	ON	ON	OFF	OFF
61	OFF	OFF	ON	ON	ON	ON	ON	ON
62	ON	OFF	ON	ON	ON	ON	OFF	OFF
63	OFF	ON	ON	ON	ON	ON	OFF	OFF
64	ON	ON	ON	ON	ON	ON	ON	ON

B.1 C Code

```
double func(int* indices, double* values)
{
    int i = indices[0];
    double v = values[i];
    v = v * v;
    return v;
}
```

x64 msvc v19.44 VS17.14 with flags /O2

```
func    PROC
        movsxd  rax,  DWORD PTR [rcx]
        movsd   xmm0, QWORD PTR [rdx+rax*8]
        mulsd   xmm0, xmm0
        ret     0
func    ENDP
```

B.2 Qspectre-load: C code shown in Appendix B.1

x64 msvc v19.44 VS17.14 with flags /O2 /Qspectre-load

```
func    PROC
        movsxd  rax,  DWORD PTR [rcx]
        lfence
        movsd   xmm0, QWORD PTR [rdx+rax*8]
        lfence
        mulsd   xmm0, xmm0
        pop     r11
        lfence
        rex_jmp r11
func    ENDP
```

B.3 /homeparams: C code shown in Appendix B.1

x64 msvc v19.44 VS17.14 with flags /O2 /homeparams

```
func    PROC
        mov     QWORD PTR [rsp+16], rdx
        mov     QWORD PTR [rsp+8], rcx
        mov     rax, QWORD PTR indices$[rsp]
        movsxd  rcx, DWORD PTR [rax]
        mov     rax, QWORD PTR values$[rsp]
        movsd   xmm0, QWORD PTR [rax+rcx*8]
        mulsd   xmm0, xmm0
        ret     0
func    ENDP
```

B.4 /GH & /Gh: C code shown in B.1

x64 msvc v19.44 VS17.14 with flags /O2 /GH /Gh

```
func    PROC
        call    _penter
        movsxd  rax, DWORD PTR [rcx]
        movsd   xmm0, QWORD PTR [rdx+rax*8]
        mulsd   xmm0, xmm0
        call    _pexit
        ret     0
func    ENDP
```

Appendix C

Blacklisted Functions

Table C.1: All IDA recognized functions that were blacklisted.

Symbol	Count
?_srt_common_main_seh@@YAHXZ	1701
?post_pgo_initialization@@YAHXZ	1701
?pre_c_initialization@@YAHXZ	1701
?pre_cpp_initialization@@YAXXZ	1701
?raw_length@charNode@@UEBAHXZ	1701
UserMathErrorFunction	1701
__C_specific_handler	1701
__GSHandlerCheck	1701
__current_exception	1701
__current_exception_context	1701
__p___argc	1701
__p___argv	1701
__p__commode	1701
__raise_securityfailure	1701
__report_gsfailure	1701
__srt_acquire_startup_lock	1701
__srt_initialize_crt	1701
__srt_initialize_default_local_stdio_options	1701
__srt_initialize_onexit_tables	1701
__srt_is_nonwritable_in_current_image	1701
__srt_is_ucrt_dll_in_use	1701
__srt_is_user_matherr_present	1701
__srt_release_startup_lock	1701
__srt_uninitialize_crt	1701
__security_check_cookie	1701
__setusermatherr	1701
_c_exit	1701
_cexit	1701
_configthreadlocale	1701
_configure_narrow_argv	1701

Symbol	Count
_crt_atexit	1701
_exit	1701
_get_initial_narrow_environment	1701
_get_startup_file_mode	1701
_guard_check_icall_nop	1701
_guard_dispatch_icall_nop	1701
_guard_xfg_dispatch_icall_nop	1701
_initialize_narrow_environment	1701
_initialize_onexit_table	1701
_initterm	1701
_initterm_e	1701
_onexit	1701
_register_onexit_function	1701
_register_thread_local_exe_atexit_callback	1701
_seh_filter_exe	1701
_set_app_type	1701
_set_fmode	1701
_set_new_mode	1701
atexit	1701
capture_previous_context	1701
ceil	1701
exit	1701
j_UserMathErrorFunction	1701
log2	1701
memcpy	1701
memset	1701
pow	1701
start	1701
terminate	1701
__alloca_probe	1573
wcslen	1029
wscmp	735
wcsrchr	735
strlen	441
__acrt_iob_func	336
__report_securityfailure	336
__stdio_common_vfprintf	336
__stdio_common_vswprintf	336
_time64	336
abort	336
capture_current_context	336

Symbol	Count
free	336
malloc	336
rand	336
srand	336
wscat_s	336
_wcsicmp	304
?file_name@__crt_win32_buffer_debug_info@@QEBAPEBDXZ	171
swprintf	147
?precision@ios_base@std@@QEBA_JXZ	135
unknown_libname_3	76
_vswprintf_c_l	42
??\$_Parse_fp_with_locale@\$0A@@?\$num_get@_WV?\$istreambuf_iterator@_WU?\$char_traits@_W@std@@@std@@@std@@CA?AU_N um_get_parse_result@1@QEADHAEAV?\$istreambuf_iterator@_WU?\$char_traits@_W@std@@@1@1AEBVlocale@1@@Z	32
?multiply_by_power_of_ten@__crt_strtox@@YA_NAEAUbig_integer@1@I@Z	16
unknown_libname_2	8

DRAFT

Appendix D

Tooling and Software Versions

D.1 Software

Software	Version
CMake	3.30.0
Python	3.13.2
IDA Pro	9.0.241217
Tigress	4.0.11
Pearl	5.32.1
Visual Studio 2022	17.14.8
cl.exe	19.44.35211.0
cl.dll	19.44.35211.0
c1xx.dll	19.44.35211.0
c2.dll	19.44.35211.0
link.exe	19.44.35211.0
mspdb140.dll	19.44.35211.0
clui.dll	19.44.35211.0
MSBuild	17.14.14 + a129329f1

Table D.1: Tools and software used with their respective versions.

D.2 Python Libraries

Library	Version
beartype	0.22.9
pefile	2024.8.26
openpyxl	3.1.5
matplotlib	3.10.8
numpy	2.3.5
bambi	0.17.2
pandas	3.0.2
arviz	0.23.4
scipy	1.17.1

Table D.2: Used Python libraries and their versions. Version number obtained with `pip show <lib>`.

DRAFT

DRAFT

